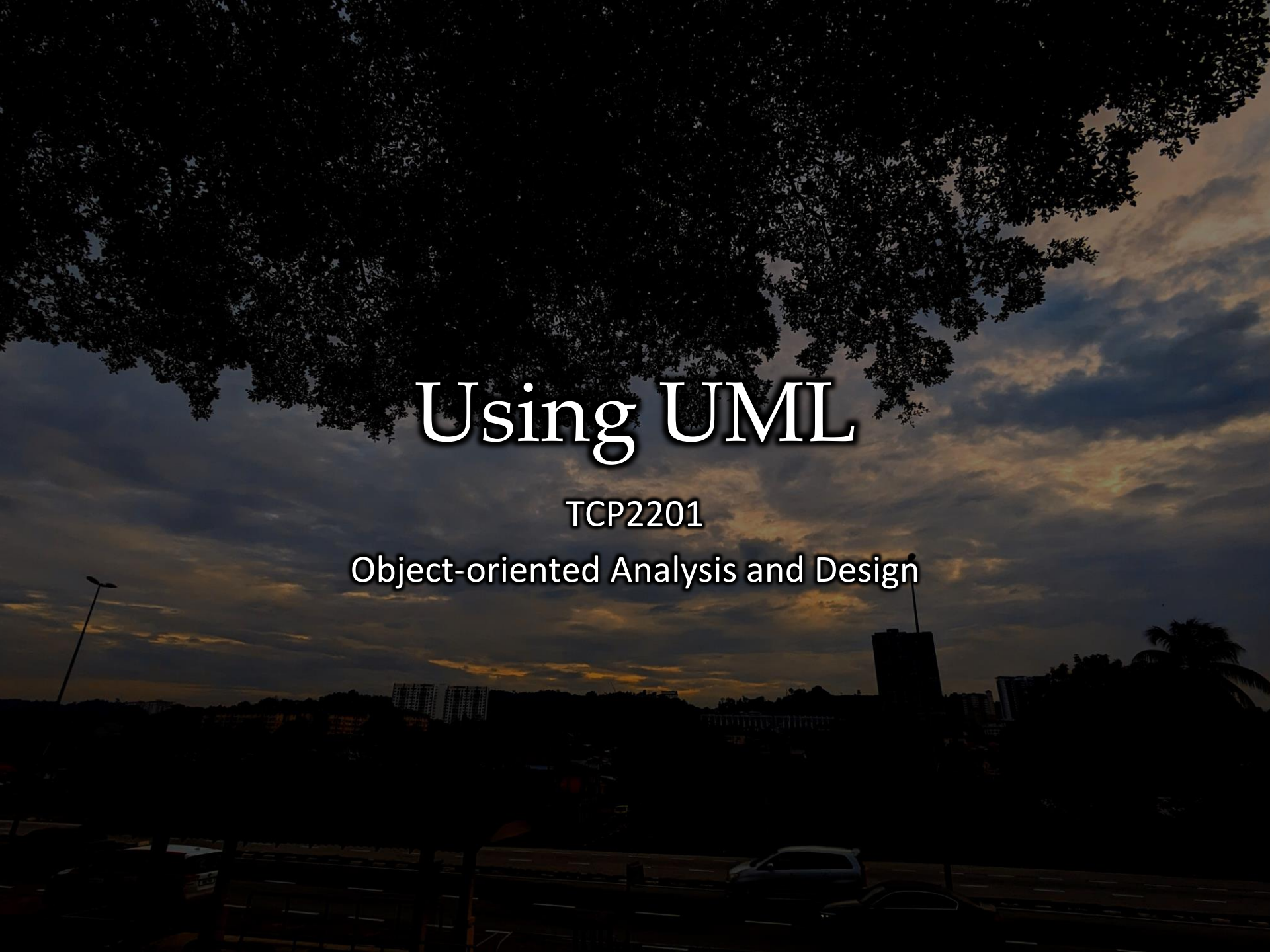


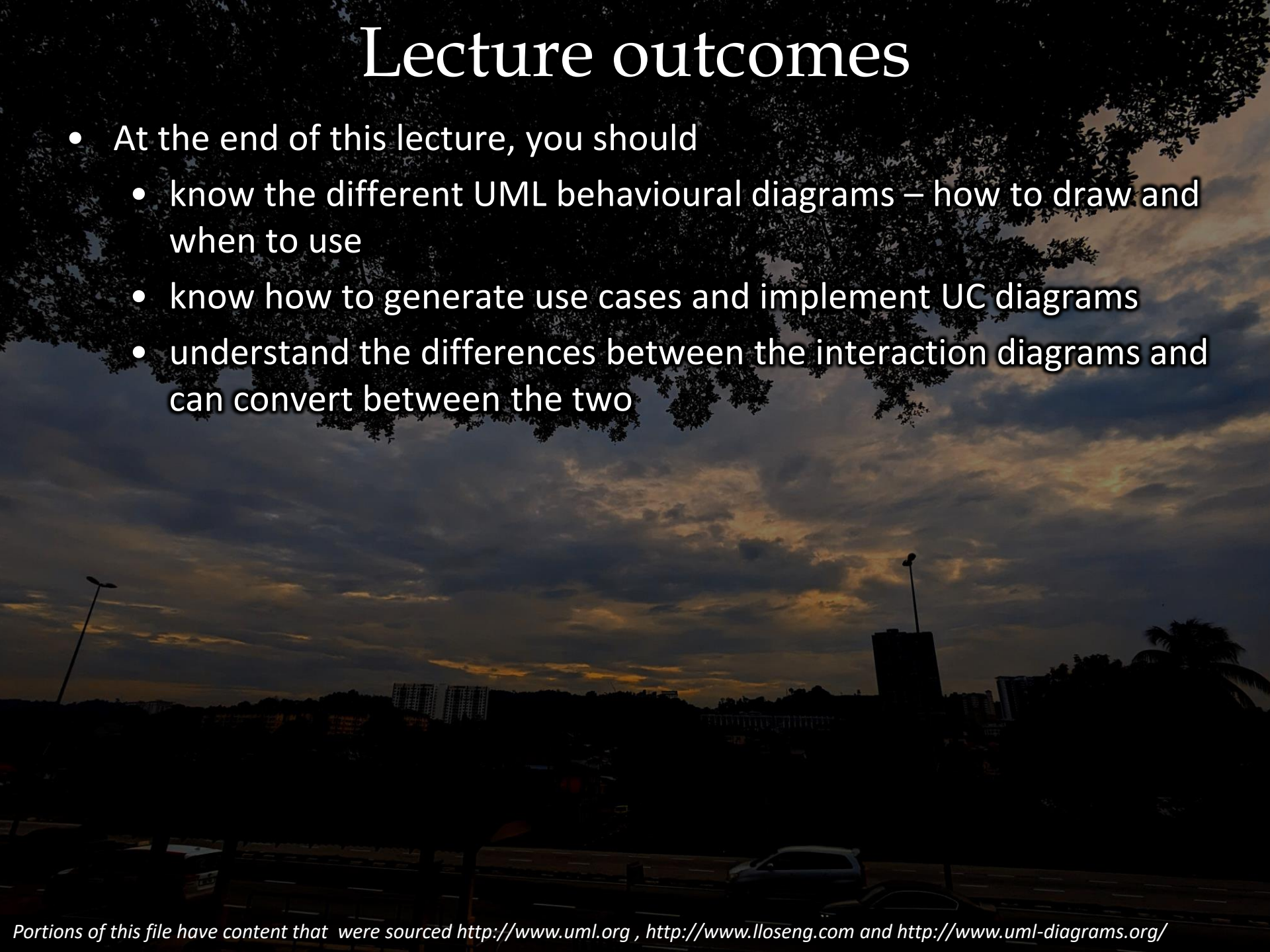
Using UML

TCP2201

Object-oriented Analysis and Design

Lecture outcomes

- At the end of this lecture, you should
  - know the different UML behavioural diagrams – how to draw and when to use
  - know how to generate use cases and implement UC diagrams
  - understand the differences between the interaction diagrams and can convert between the two



Portions of this file have content that were sourced http://www.uml.org , http://www.lloseng.com and http://www.uml-diagrams.org/

Recap

Learned previously in OOPDS:

- Structural Diagrams
  - Focuses on static aspects of the software system
    - Class, Object, Component, Deployment

Today we will focus on:

- Behavioral Diagrams
  - Focuses on dynamic aspects of the software system
    - Use-case, Interaction, State, Activity

UML use cases

- A Use Case Model describes the proposed functionality of a system.
- Each UC represents a discrete unit of interaction between a user and the system.
- This interaction is a single unit of meaningful work, such as Create Account or View Account Details.
- Users can be ‘human’ or another system.
- Each UC describes the functionality to be built in the proposed system, which can include *another* UC's functionality or extend another UC with its own behaviour.
- The UCs in a system or subsystem can be presented in textual form but is usually depicted graphically using the use case diagram

Use cases

- A complete UC description will generally include:
  - General comments and notes describing the use case.
  - Requirements - The formal functional requirements of things that a UC must provide to the end user that correspond to the functional specifications
  - Constraints - The formal rules and limitations a UC operates under, defining what can and cannot be done. These include Pre-conditions, Post-conditions and Invariants
  - Scenarios – Formal, sequential descriptions of the steps taken to carry out the use case, or the flow of events that occur during a Use Case instance. These are usually created in text and correspond to a textual representation of the Sequence Diagram.
  - Scenario diagrams - Sequence diagrams to depict the workflow; similar to Scenarios but graphically portrayed.
  - Additional attributes, such as implementation phase, version number, complexity rating, stereotype and status.

Use case example

- The following scenario is used as an example to generate use cases

1. A housekeeper does laundry on a Wednesday
2. She washes each load
3. She dries each load
4. She folds certain items
5. She irons some items
6. She throws away certain items

Example based on source at http://www.usability.gov/how-to-and-tools/methods/use-cases.html

Use case example 1

- Basic use case

Use Case 1	Do laundry
Actor	Housekeeper
Basic Flow	On Wednesdays, the housekeeper reports to the laundry room. She sorts the laundry that is there. Then she washes each load. She dries each load. She folds the items that need folding. She irons and hangs the items that are wrinkled. She throws away any laundry item that is irrevocably shrunken, soiled or scorched.

- Extended use case

Use Case 1	Do laundry
Actor	Housekeeper
On Wednesdays, the housekeeper reports to the laundry room. She sorts the laundry that is there. Then she washes each load. She dries each load. She folds the items that need folding. She throws away any laundry item that is irrevocably shrunken, soiled or scorched.	
Alternative Flow 1	If she notices that something is wrinkled, she irons it and then hangs it on a hanger.
Alternative Flow 2	If she notices that something is still dirty, she rewashes it.
Alternative Flow 3	If she notices that something shrank, she throws it out.

UC templates

- Not really standardized …. so here are few valid examples

Use case ID:	
Use case name:	
Description:	
Pre conditions:	
Standard flow:	
Alternate flow:	
Post conditions:	
Open issues:	

UC-1: Customer logs in and finds a free spot.

Description/Story:

PreCondition:

PostCondition:

Actor(s): All Actors Assigned | **Package:** No Package Assigned

Status: Submitted | **Priority:** High | **Rank:** 0.0

Assigned To: Unassigned | **Created by:** darrenjlevy@gmail.com | **Created Date:** 2011-08-1

Step:	User Action:	System Response:
1	User opens up app	App shows credentials screen
2	User types in credentials	App shows current location on a mapp
3	User selects they are looking for a spot	App accepts info
4		App generates the updated screen with live spaces
5	Customer clicks on a live available space	App shows details including time of departure
6	Customer accepts free spot and updates	App clears out available spot.

Req ID:	Requirement Name:	Priority:	Created	
BR1	System shall allow a user to log in to the app	High	2011-08	
BR3	System shall provide a map of offered spots in real-time	High	2011-08	
BR5	Providers can set a price for their spot	High	2011-08	
BR6	Provider can change their spot from fee to paid and vice versa	High	2011-08	
BR7	Searcher can offer to the provider to hold a spot for a given time	High	2011-08-14	Not Assigned
BR8	Provider can reject a bid	High	2011-08-14	Not Assigned
BR11	Users can fund account via credit card	High	2011-08-14	Not Assigned
BR13	Customer can transfer money from account to their checking account	High	2011-08-14	Not Assigned

USE CASE <Use case code>	<Use case name>	Date:	<date of the last change >
		Version:	<use case version>

Description:	<use case brief description>	
User priority:	<development priority requested set by users>	
Performance:	<requested performance>	
Primary actor:	<primary actor name> <primary actor interest in this service>	
Secondary actor:	<secondary actor name> < secondary actor interest in this service>	
Preconditions:	<description of the conditions which must be fulfilled before the use case can be executed>	
Post-condition	on success:	<description of the conditions that describe the state of a system after that this use case is successfully completed>
	on failure:	<description of the conditions that describe the state of a system after that this use case is unsuccessfully completed>
Trigger:	<description of the event that fires the use case>	

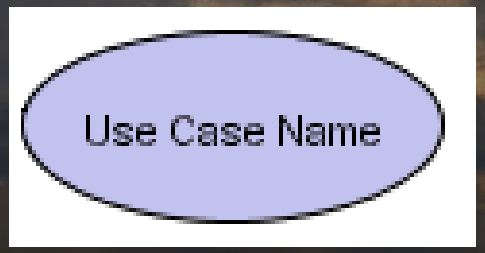
MAIN SCENARIO		
1.	<actor/ system>	<action description>
2.	<actor/ system>	<action description>
…	<actor/ system>	<action description>
n.	System:	Terminates use case successfully.
ALTERNATIVE SCENARIO: <condition which leads to this alternative scenario>		
h.1.	<actor/ system>	<action description>
		<action description>
h.n.	System:	Resumes use case at point x.y.
EXCEPTION SCENARIO: <condition which leads to this exception scenario>		
i.1.	<actor/ system>	<action description>
		<action description>
i.m.	System:	Terminates use case unsuccessfully.
ANNOTATION		
	<extra information>	

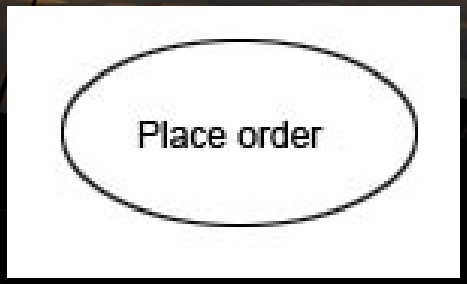
Use case diagrams

- Use case diagrams depict use cases graphically with the following components:
  - Use cases
  - Actors
  - Associations
  - System boundary boxes (optional)
  - Packages (optional)

UCD use cases

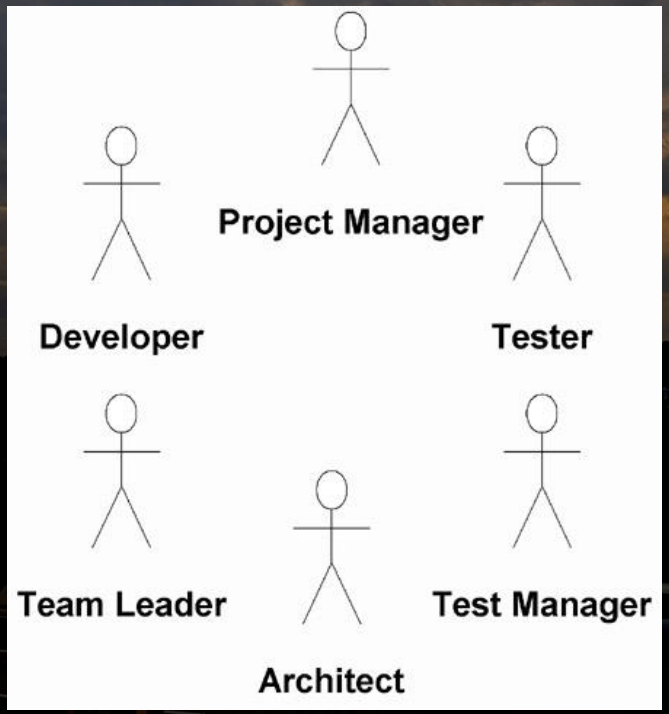
- A use case describes a sequence of actions that provide something of measurable value (responsive action) to an actor
- Drawn as a horizontal ellipse with the name of the UC within
- Most UCs make use of verb-like descriptions (e.g. input student ID, make call. destroy object etc)
- Descriptions in UCs should be short statements and not long sentences





UCD actors

- An actor is a person, organization, or external system that plays a role in one or more interactions with your system
- Actors are represented using stick figures and represent the users from the use case
- Actors can be humans that interact with the system (users) or other systems that work with the system (e.g. client-server)

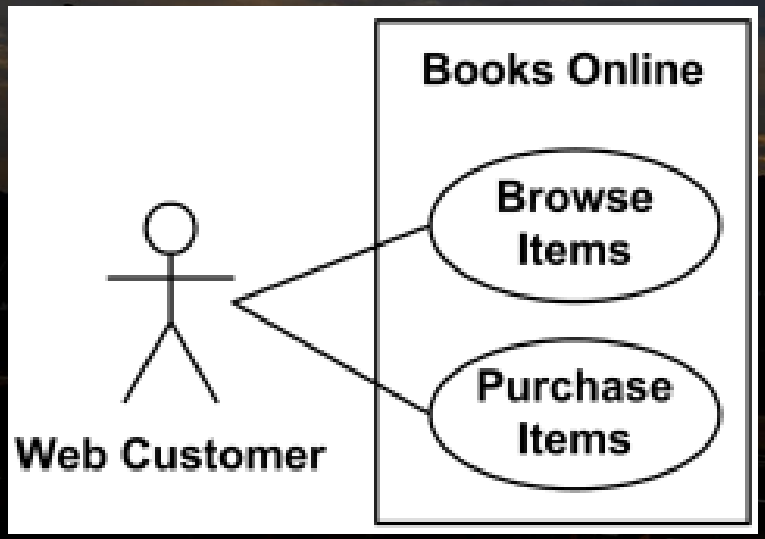


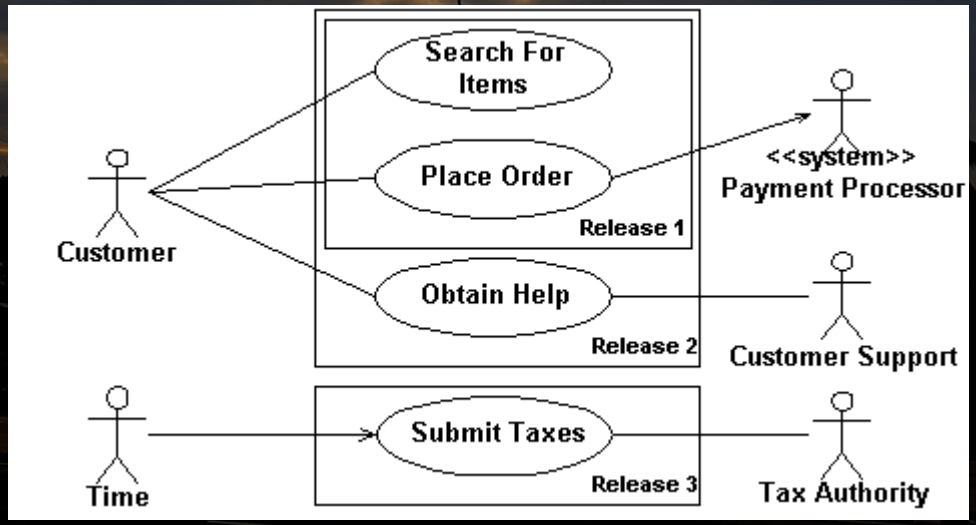
UCD associations

- An association (or communication) exists whenever an actor is involved with an interaction described by a use case
- Associations are modeled as lines connecting use cases and actors to one another, with an optional arrowhead on one end of the line.
- The (optional) arrowhead is used to indicate the direction of the initial invocation of the relationship or to indicate the primary actor within the use case (i.e. who invokes who)

UCD system boundary

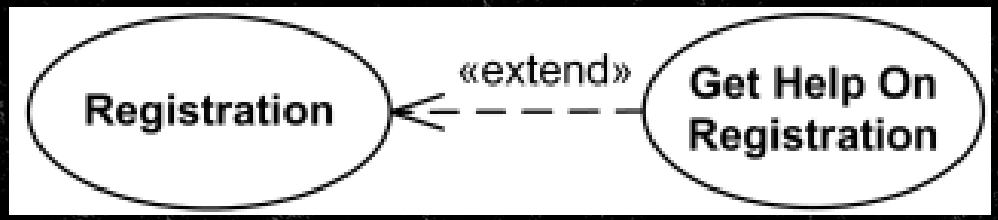
- A rectangle can be drawn around the use cases, called the system boundary box, to indicate the scope of your system.
- Anything within the box represents functionality that is in scope and anything outside the box is not
- Actors usually are external to the system and only interact with UCs within the boundary (through the system interface)
- System boundaries can be labeled to show system name
- It is also possible to use boundaries to identify which use cases will be delivered in each major release of a system

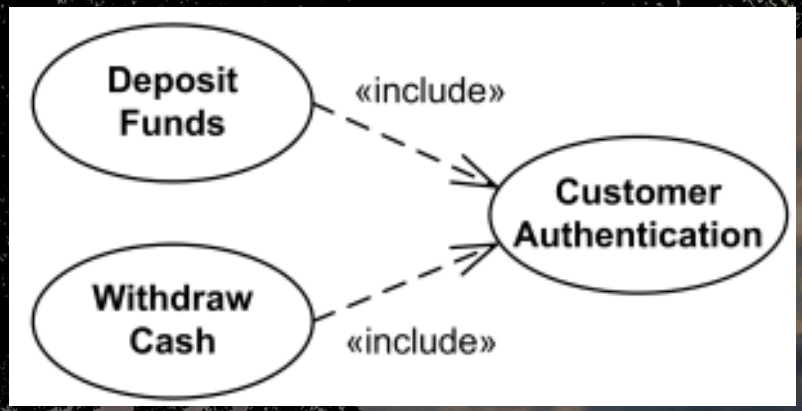


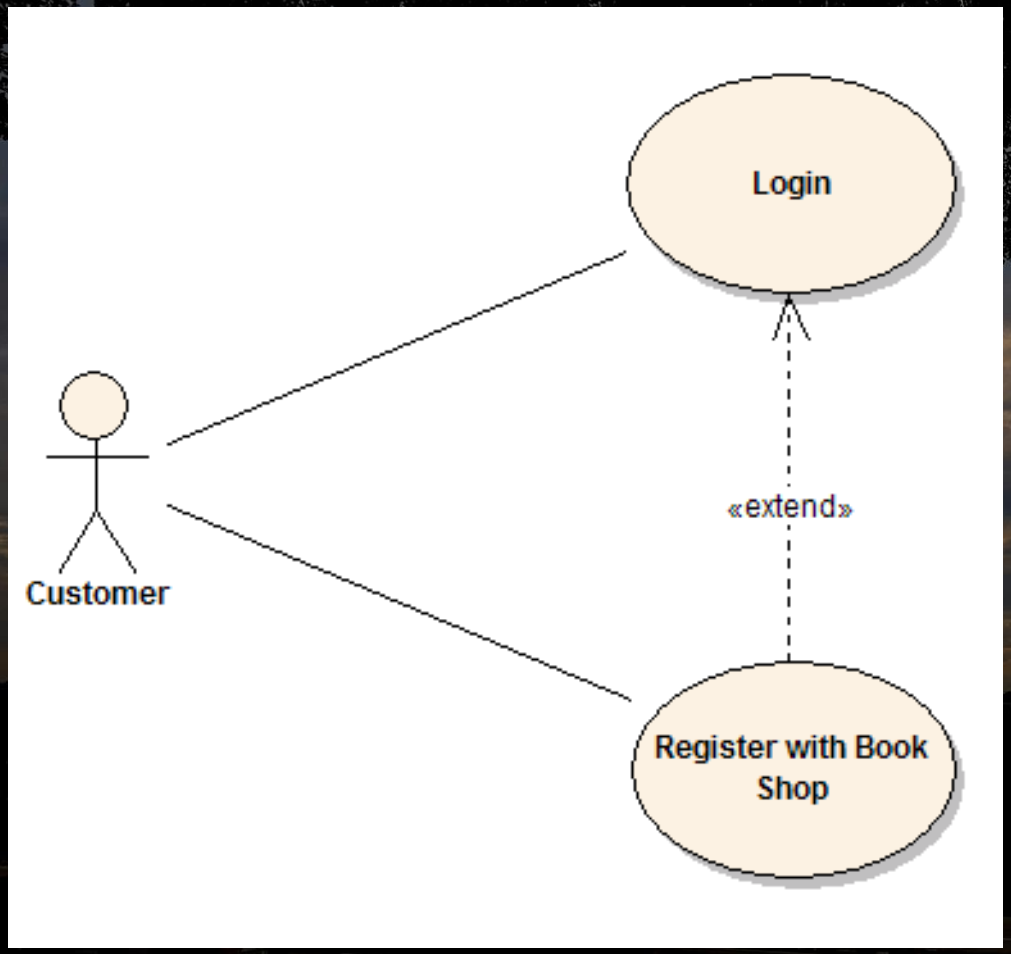


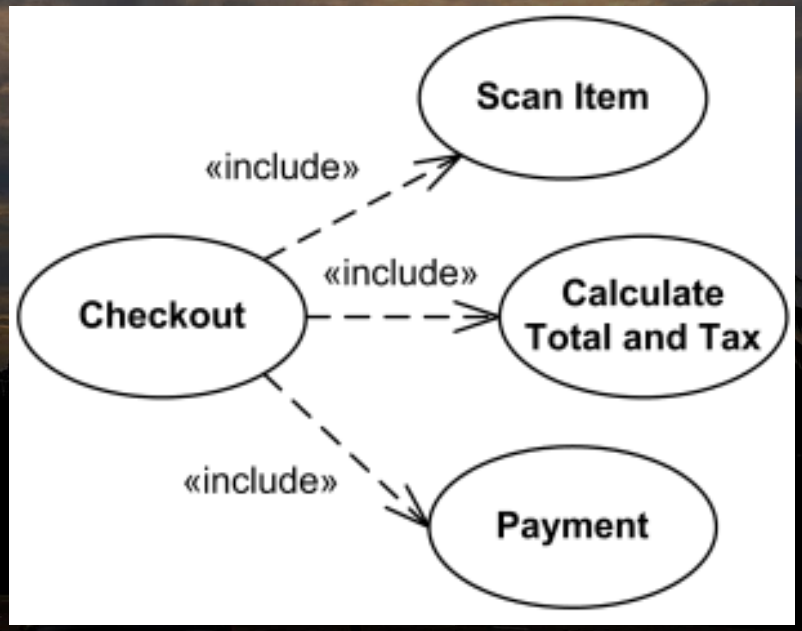
UCD extend and include

- One UC could *include* the functionality of another as part of its normal processing.
  - For example, when a customer wants to view their web orders, the <login> UC would be included every time the <list order> UC is run.
- A UC can be included by one or more other UCs – this helps to reduce duplication of functionality by factoring out common behavior into UCs that are re-used many times.
- UCs can extend the behavior of another, typically when exceptional circumstances are encountered.
  - For example, if a user must get approval from some higher authority before modifying a particular type of customer order, then the <get approval> UC could optionally extend the regular <modify order> UC.
- Both extend and include make use of a dotted line notation between UCs as well as a tag to indicate its type


Registration
«extend»
Get Help On Registration

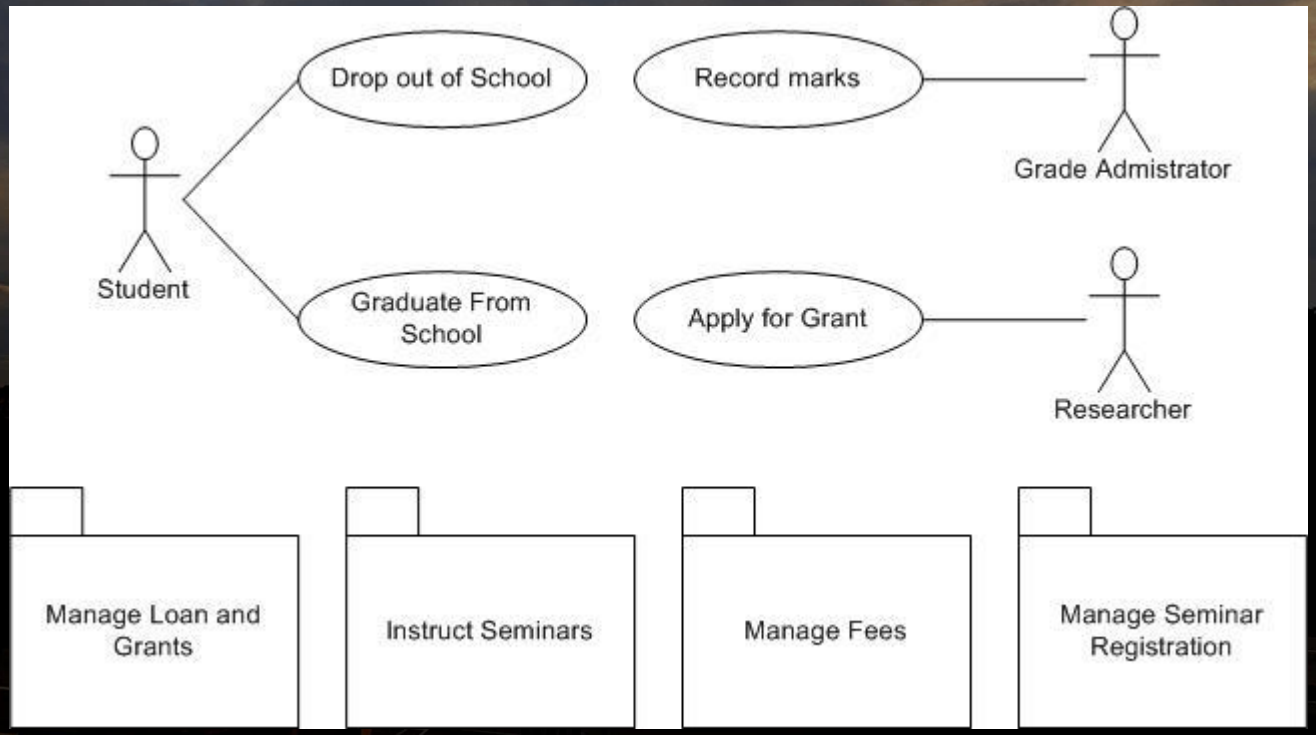

Deposit Funds
«include»
Customer Authentication
Withdraw Cash
«include»

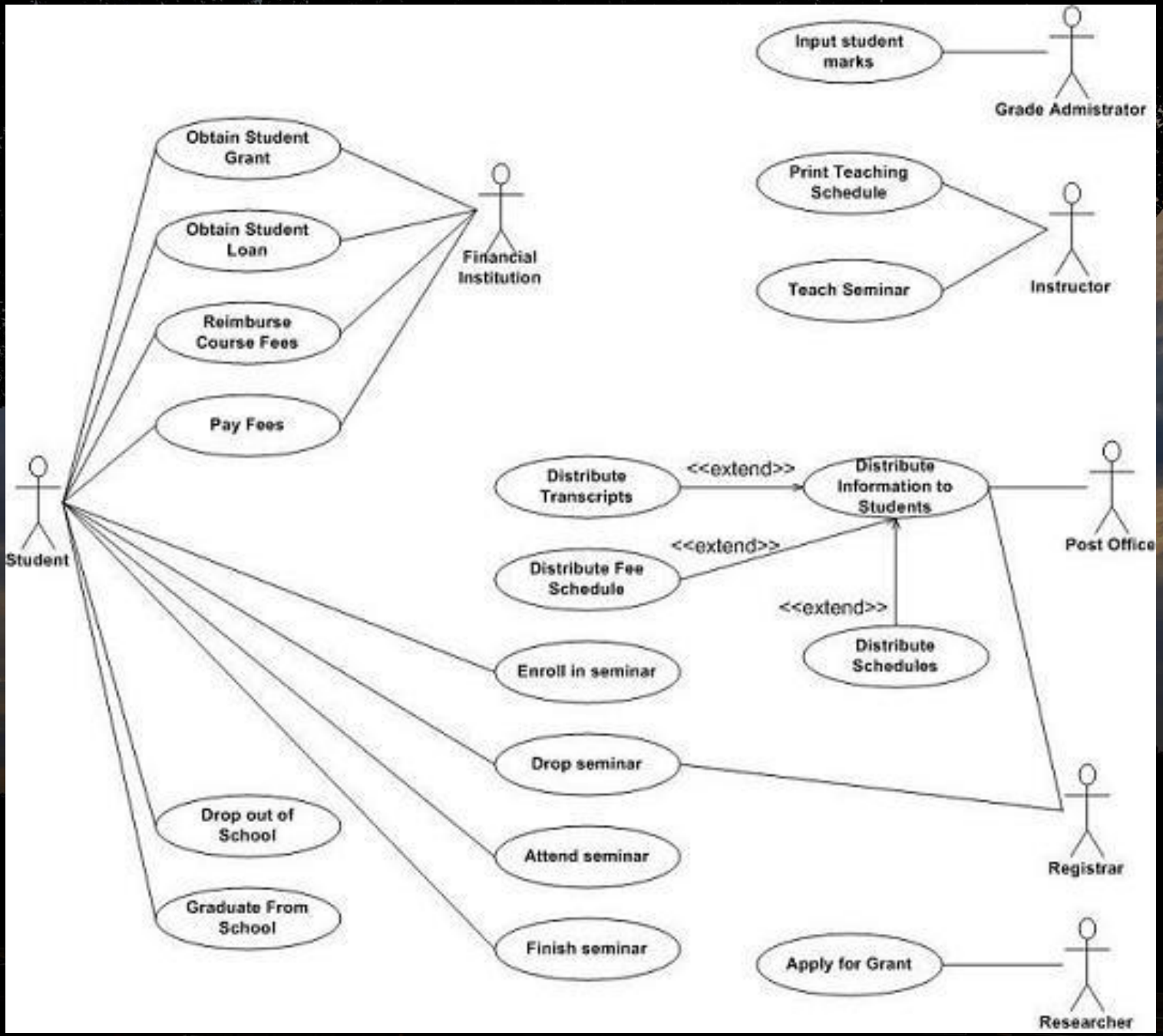

Login
Customer
«extend»
Register with Book Shop


Scan Item
«include»
Checkout
«include»
Calculate Total and Tax
«include»
Payment

UCD packages

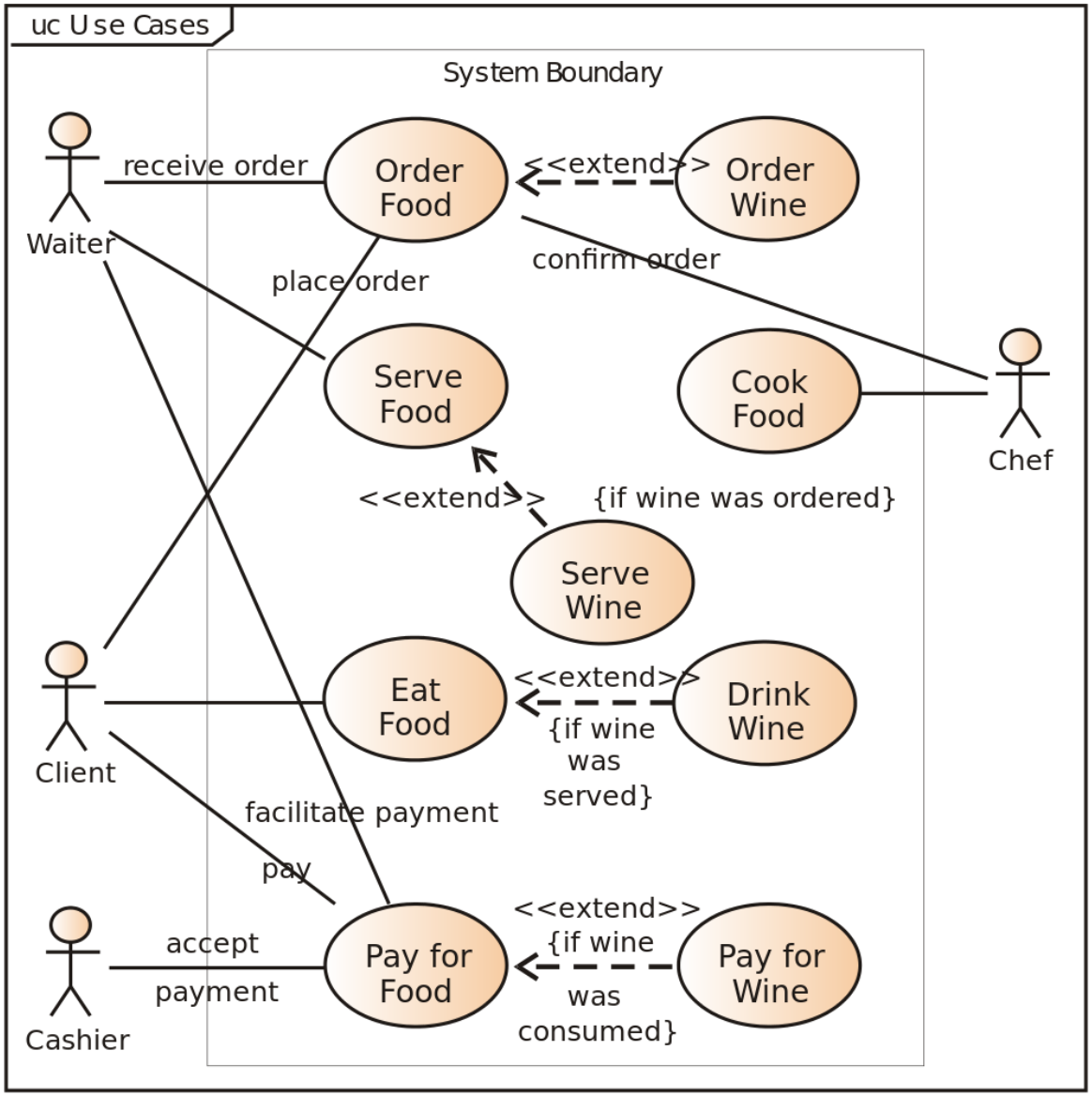
- Packages are UML constructs that enable you to organize model elements (such as use cases) into groups to simplify a diagram that has become too complex with multiple UCs.
- Packages are depicted as file folders and can be used on any of the UML diagrams, including both use case diagrams and class diagrams.
- The packages below simplify the UC diagram on the next slide





Source from http://www.agilemodeling.com/artifacts/useCaseDiagram.htm

UCD example 2



UML interaction diagrams

- Interaction diagrams are used to model the dynamic aspects of a software system
- Often built from a use case and a class diagram → to show how a set of objects accomplish the required interactions with an actor
- Interaction diagrams show how a set of actors and objects communicate with each other to perform the steps of a use case, or the steps of some other piece of functionality (i.e. interaction)
- Two types:
    - Sequence Diagram
    - Collaboration Diagram
- Difference between Sequence and Collaboration → how the sequence of message calling are illustrated

UML sequence diagram

- The UML sequence diagram is an interaction diagram that emphasizes the time ordering of messages
- It shows a set of objects and the messages sent and received by those objects over a period of time
    - That is, sequence diagrams are used to model object interactions arranged in time sequence and to distribute use case behavior to classes.
    - They can also be used to illustrate all the paths a particular use case can ultimately produce
- Graphically, the diagram is a table that shows objects arranged along the X axis and messages, ordered sequentially on the Y axis
- Components of the diagram include (active) Objects, Messages and Life-lines

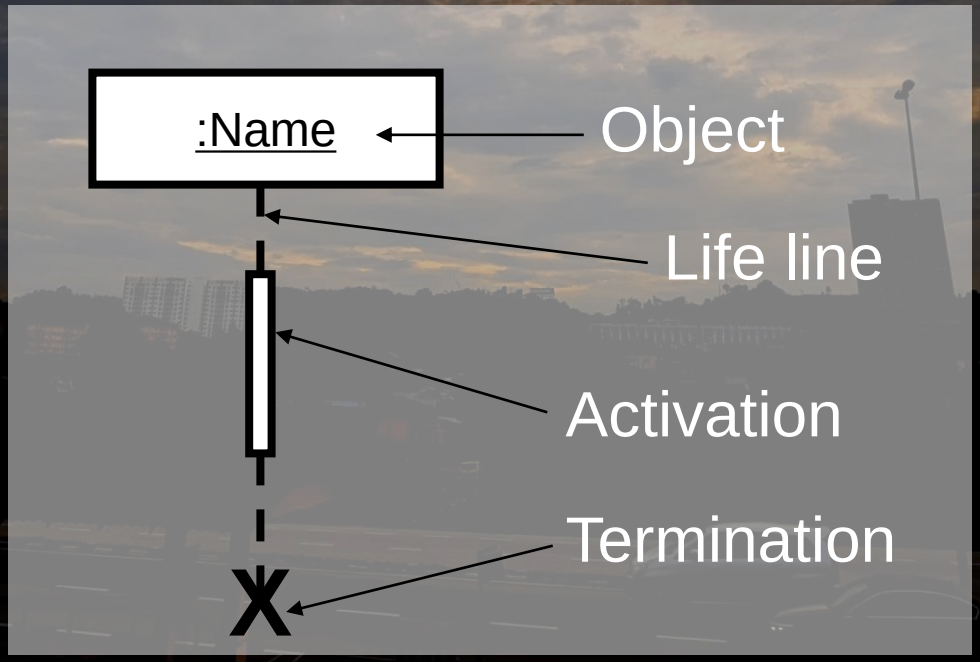
Seq. diagram objects

- An object in a sequence diagram is rendered as a box
- The box may be labeled with object class names or instance variables
- Object identifiers are of the form objectName:className and either the objectName or the className can be omitted, and the placement of the colon indicates either an objectName: or a :className

- If an interaction requires a third/external party, the actor notation is sometimes added for clarity

Seq. diagram life-lines

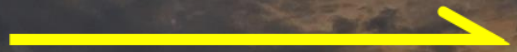
- Object in a sequence diagram have a dashed line descending from it representing the timeline/life-line of the object
- A life-line shows what happens to an object in a chronological order
- The life-line is drawn for the period of time the object remains valid in the interaction being depicted → ends with X
- Any activity/involvement of the object in an interaction can be shown with activation boxes (optional)



Seq. diagram messages

- Messages are passed between objects in the sequence diagram
- When a message is passed, it creates an activation box on the source and destination object for the period the message is active
- There are 4 general types of messages each with its own notation
  - Synchronous → interrupt flow until message completed

  - Asynchronous → don't wait for response, just return to caller

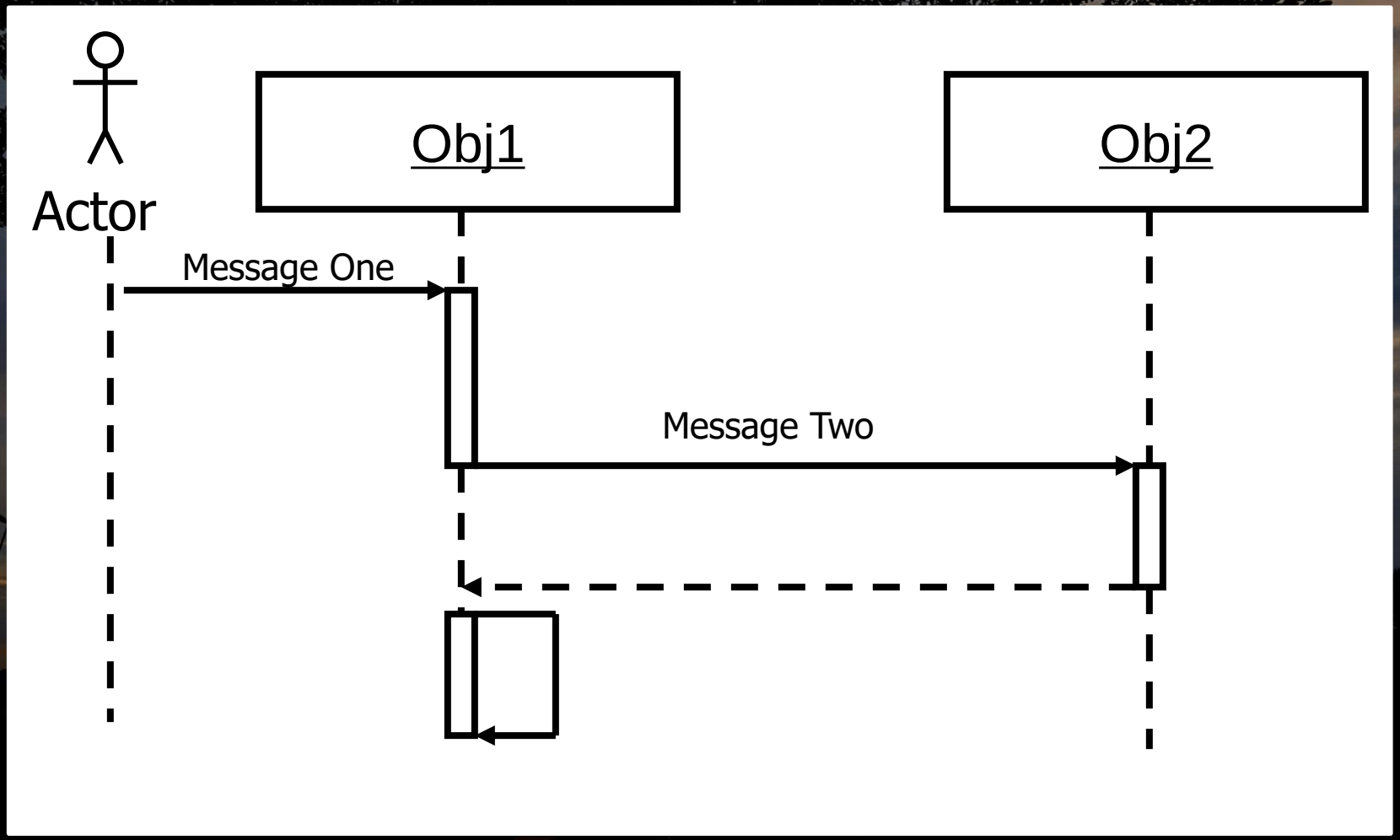


  - Flat → no distinction between sysn/async or unknown

  - Return → control flow has returned to the caller

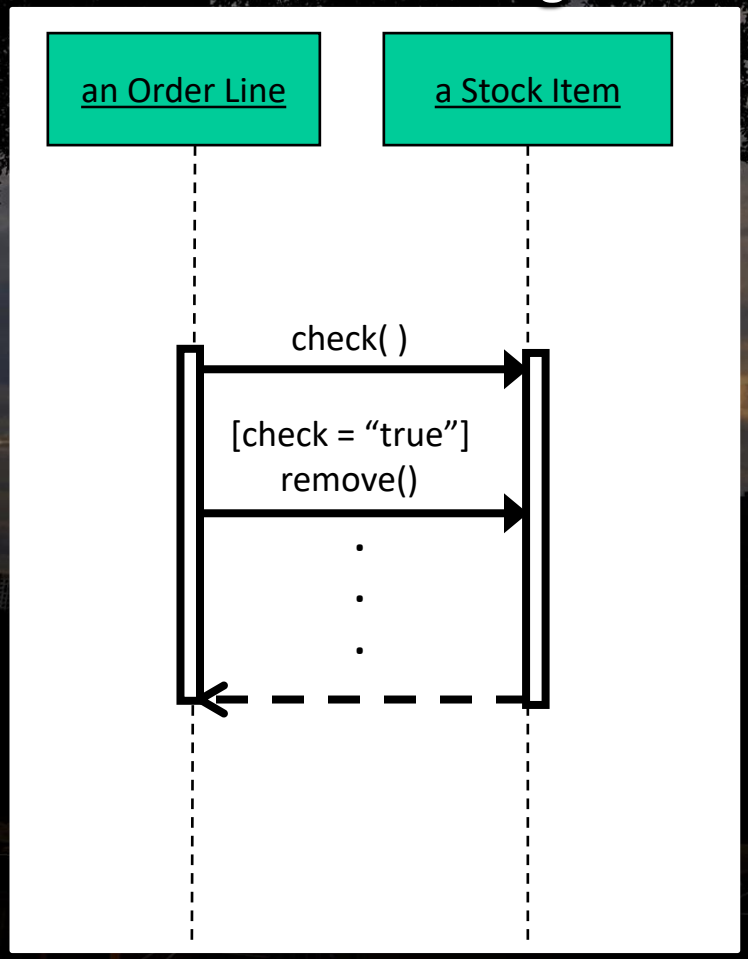
UML sequence diagram

- Combining the components creates a basic sequence diagram

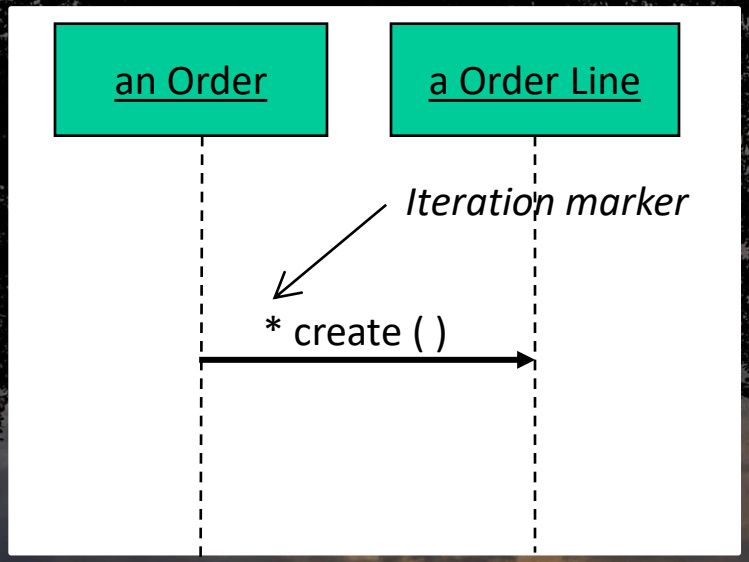


Seq. diagram message conditions

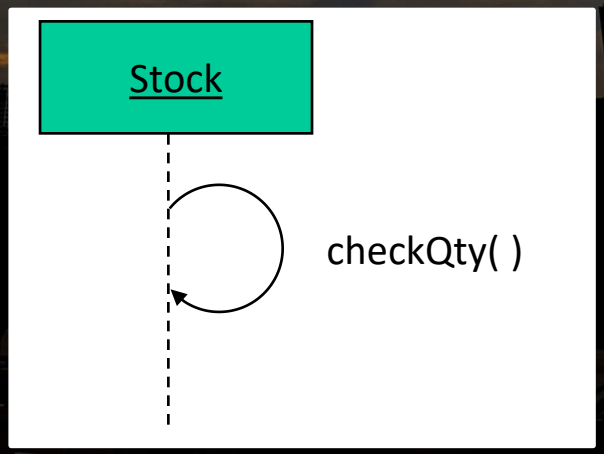
- Messages are rendered as horizontal arrows being passed from object to object as time advances down the object lifelines.
- Conditions (such as [check = “true”]) indicate when a message gets passed or what gets returned in a message



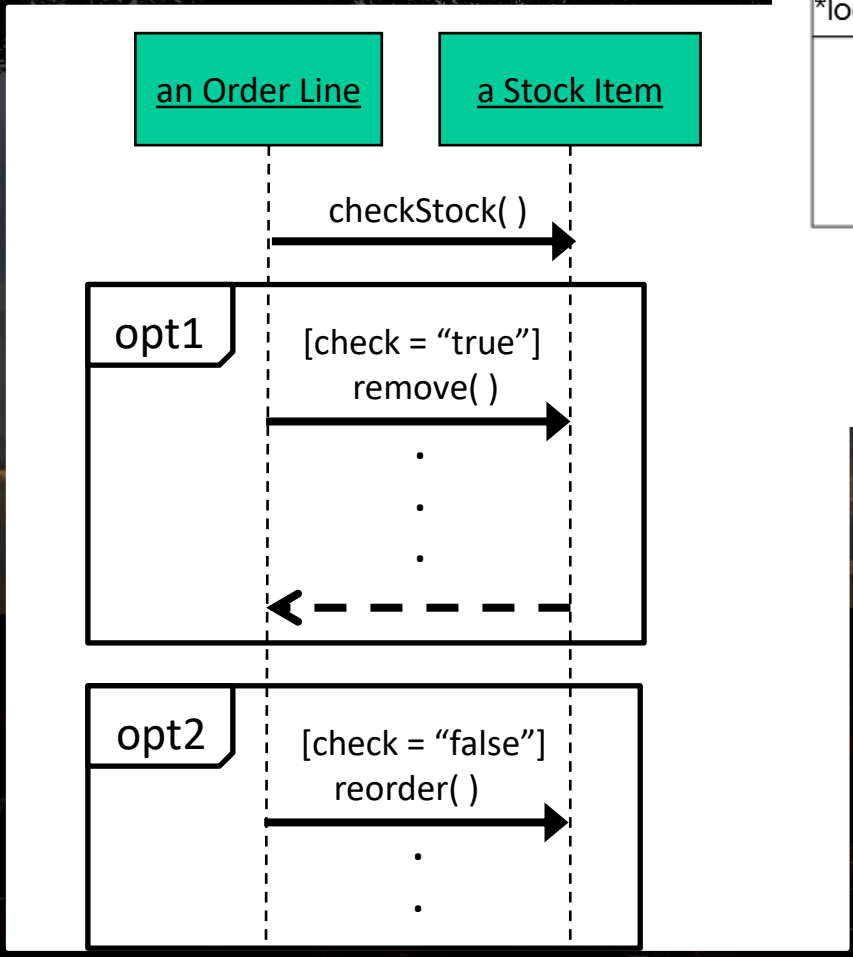
- An iteration marker, such as * or *[i = 1..n] , indicates that a message will be repeated as indicated

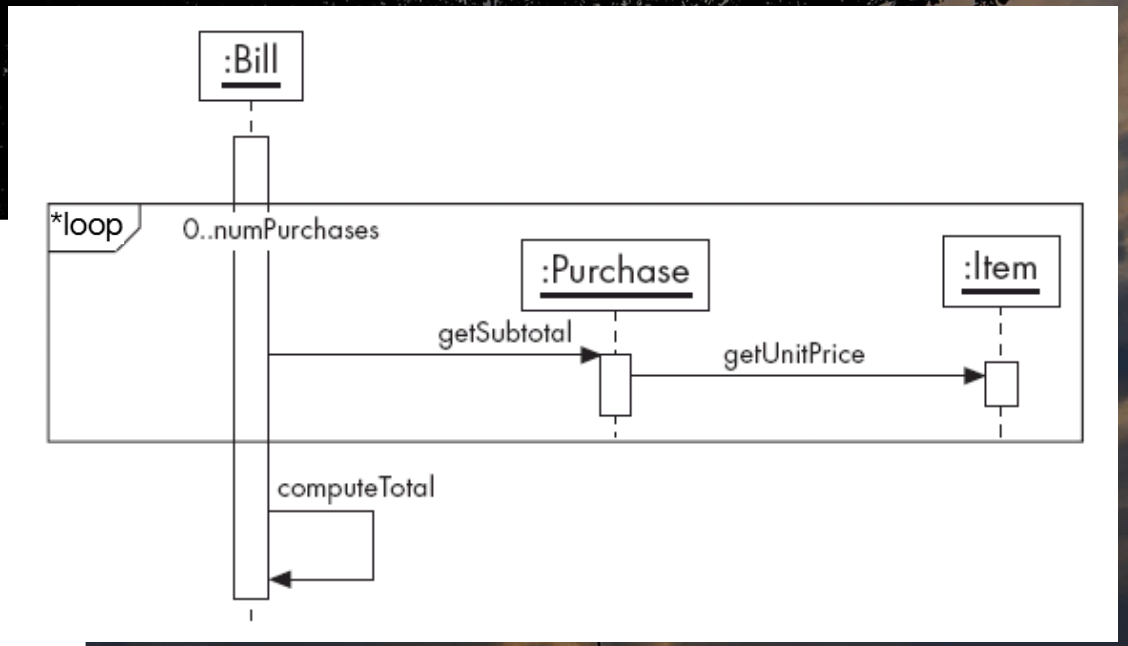


- Self-delegation can be indicated with a arrow pointing back to the same life-line (i.e. recursive call)



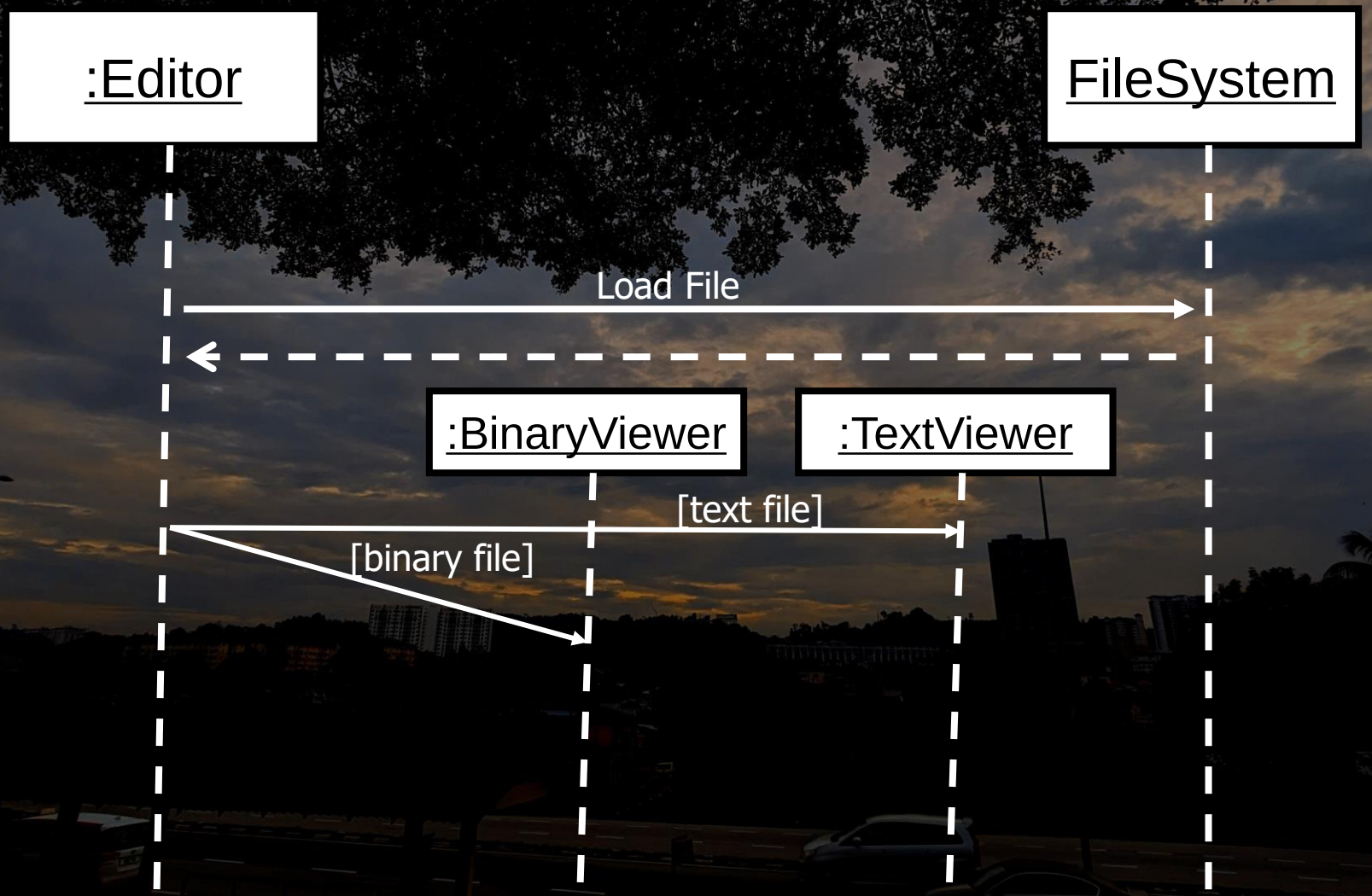
- Labeled boundaries can be used to indicate options and/or multiple messages that are looped



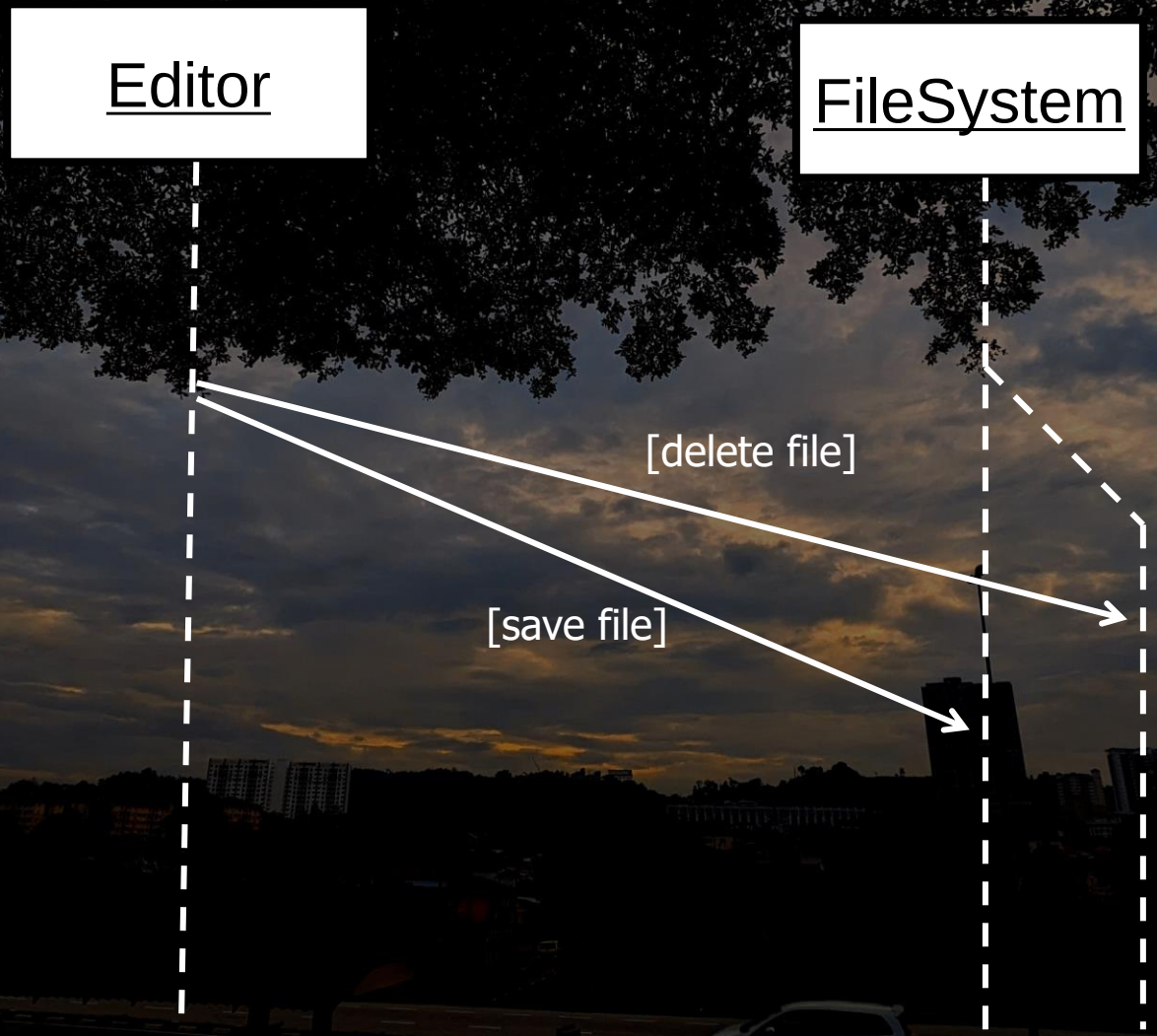


Seq. diagram branching

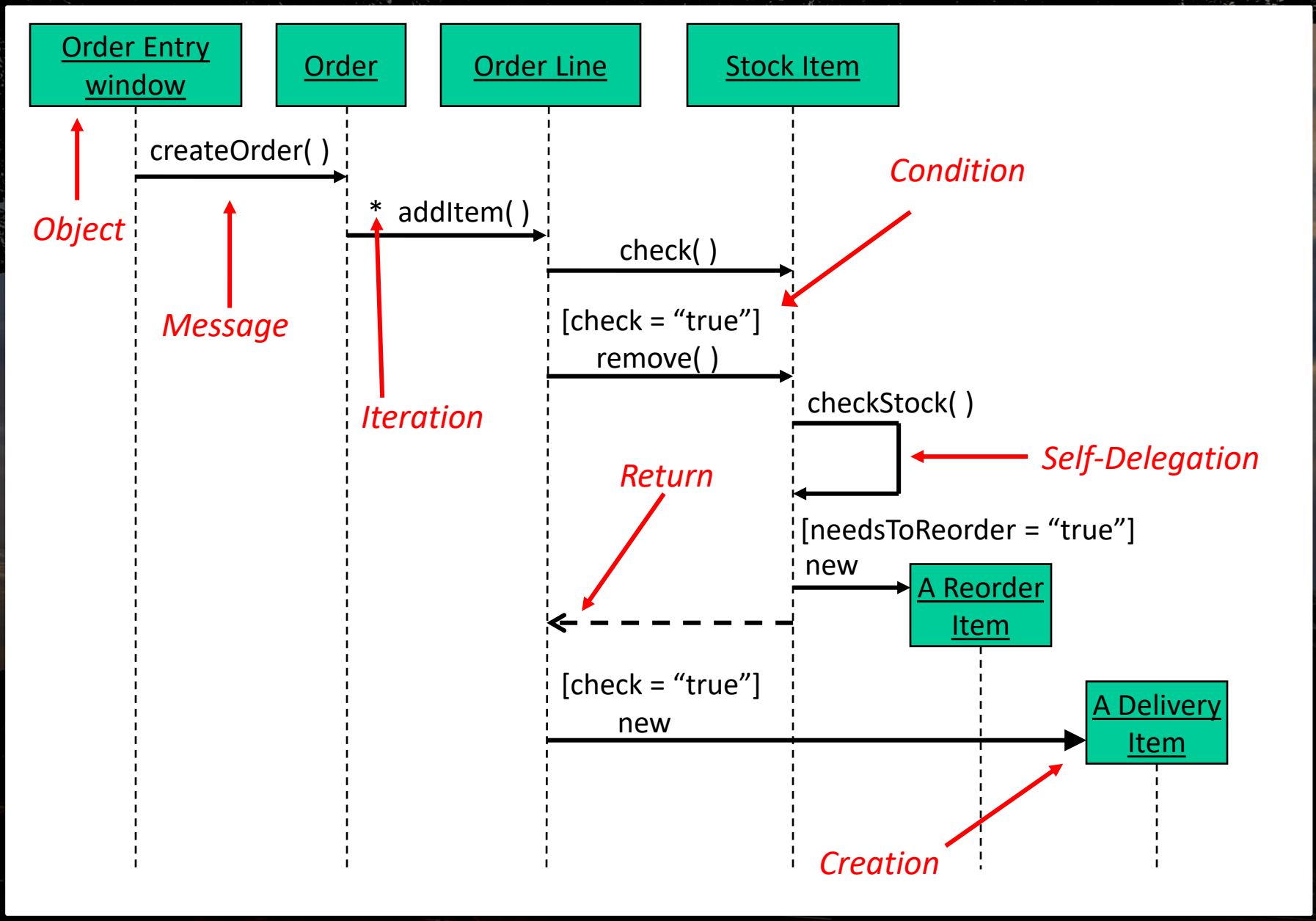
- A message may *branch* to two (or more) possible objects depending on if a condition is met or not



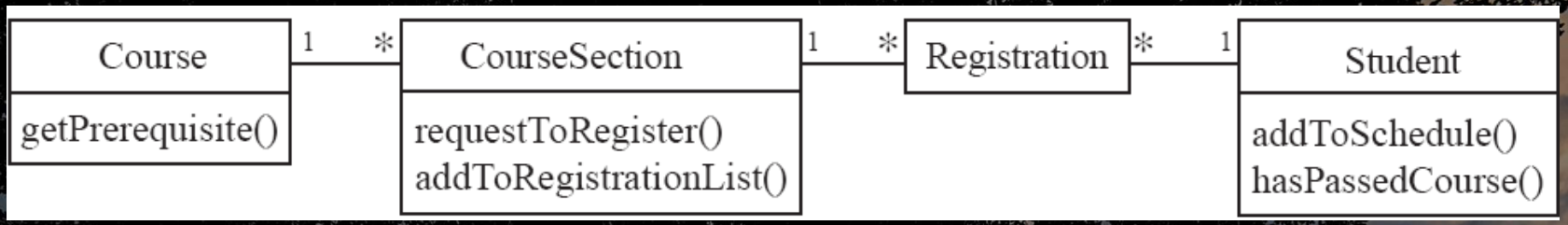
- Alternatively, an object itself may have more than one life-line and may branch of depending on conditions

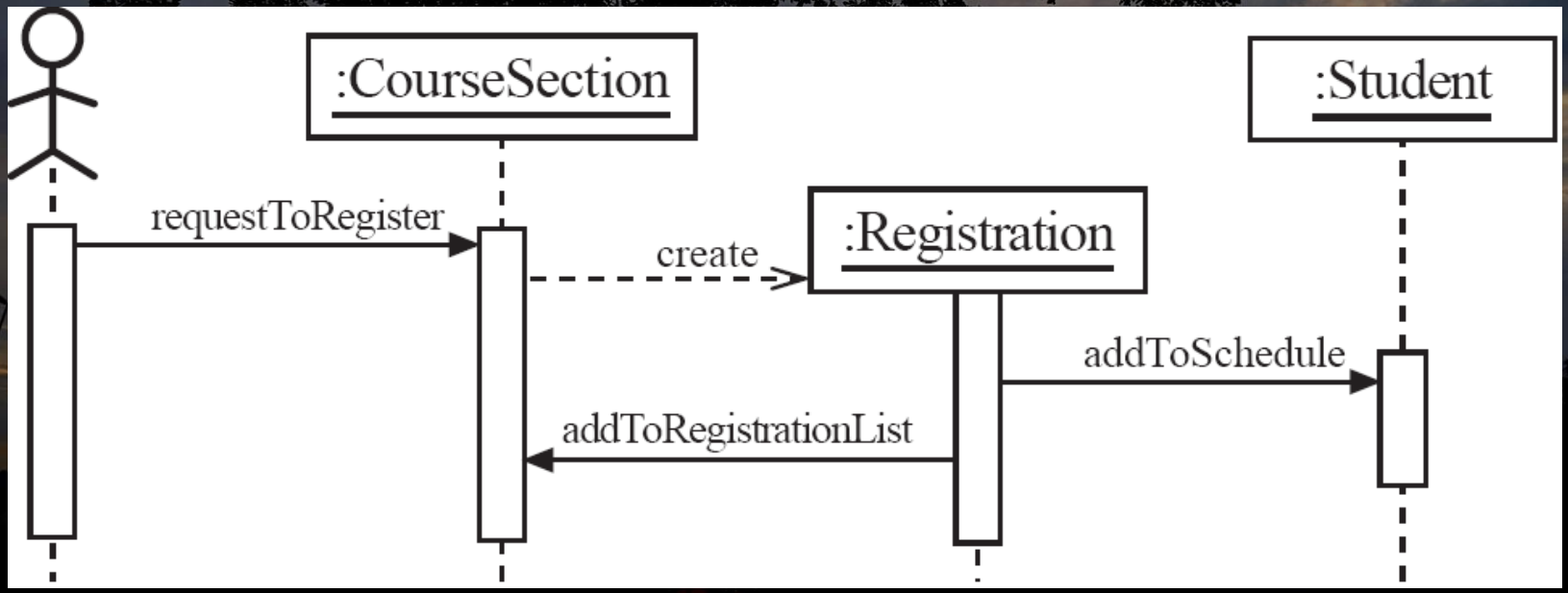


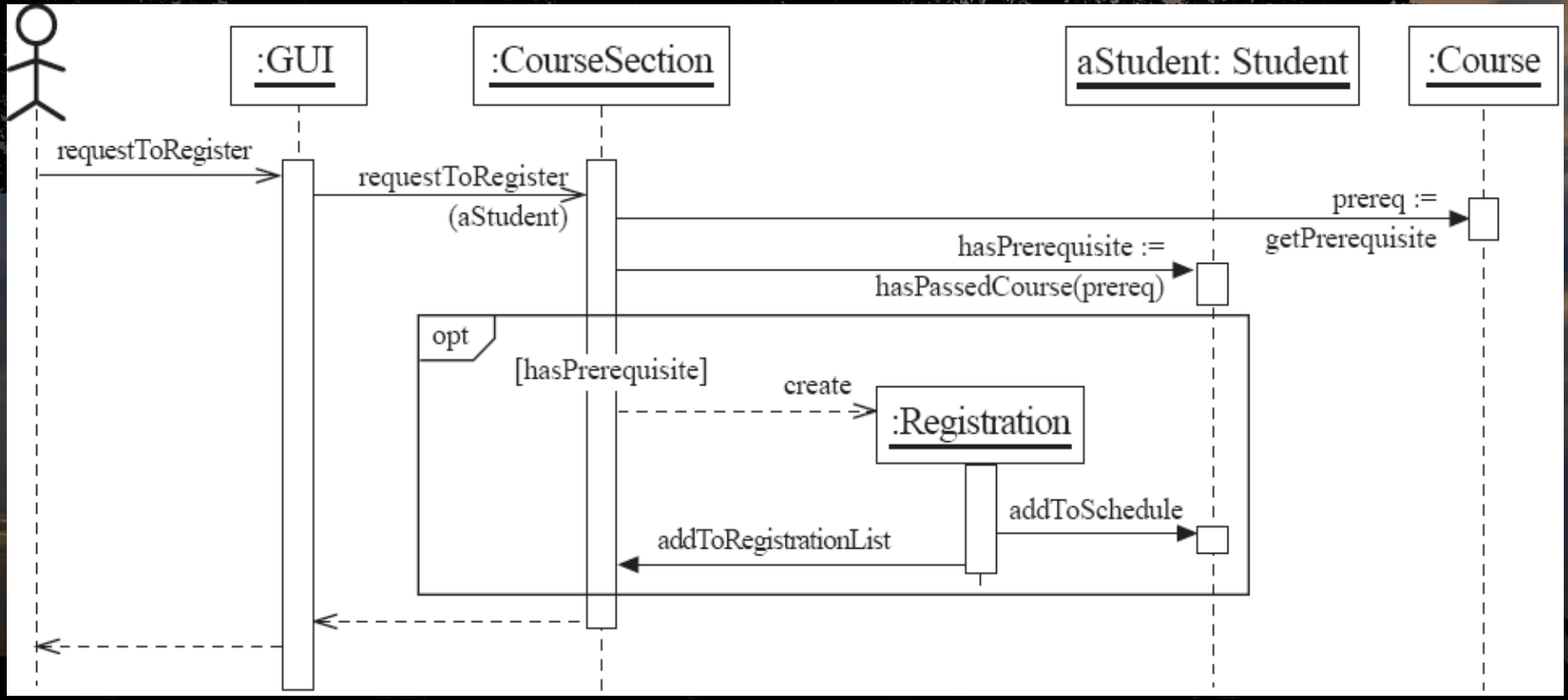
Example


Order Entry window
Order
Order Line
Stock Item
createOrder( )
* addItem( )
check( )
[check = "true"]
remove( )
checkStock( )
[needsToReorder = "true"]
new
A Reorder Item
[check = "true"]
new
A Delivery Item
Object
Message
Iteration
Condition
Self-Delegation
Return
Creation

Example 2

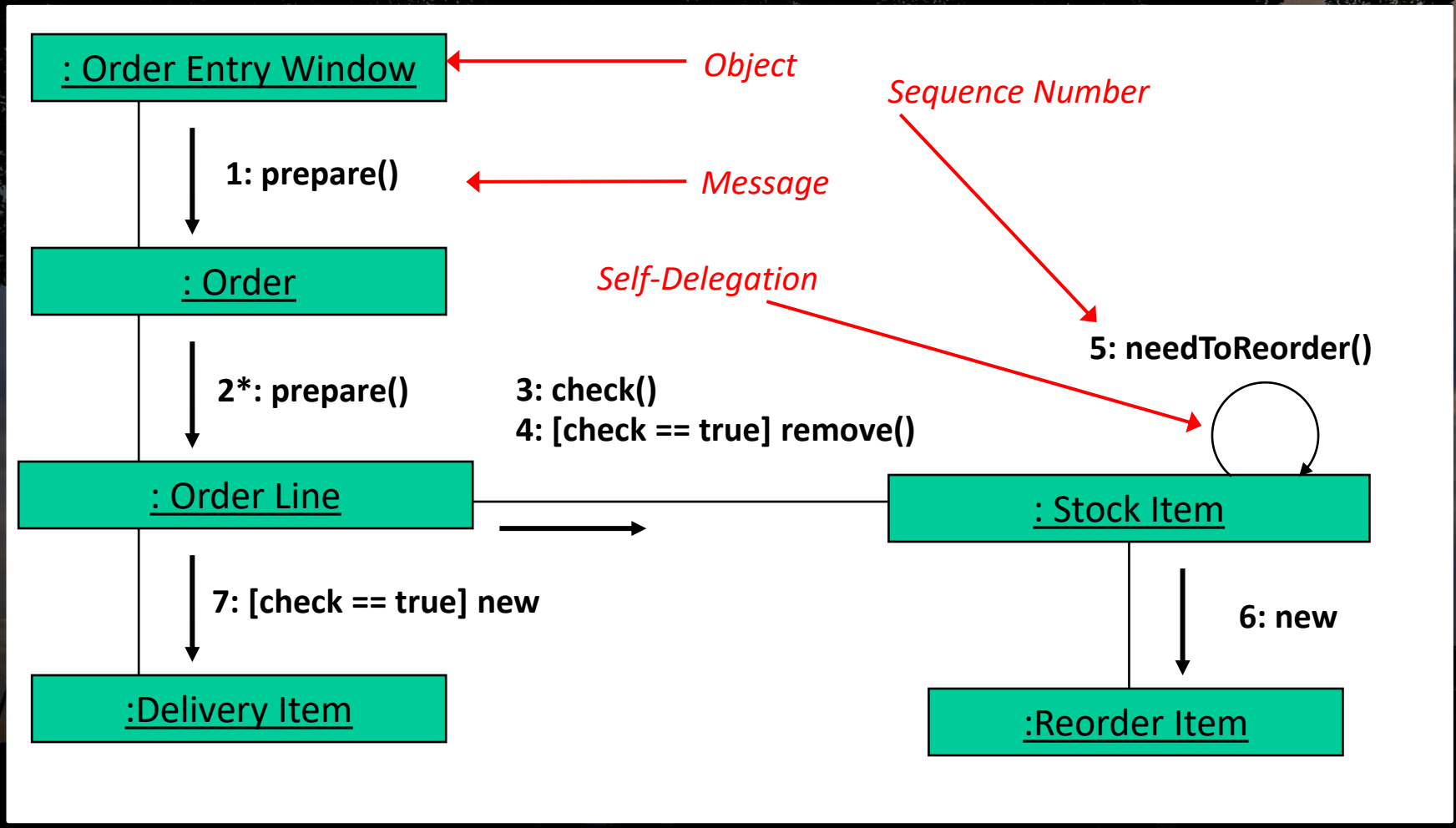

Course
getPrerequisite()
1
*
CourseSection
requestToRegister()
addToRegistrationList()
1
*
Registration
*
1
Student
addToSchedule()
hasPassedCourse()

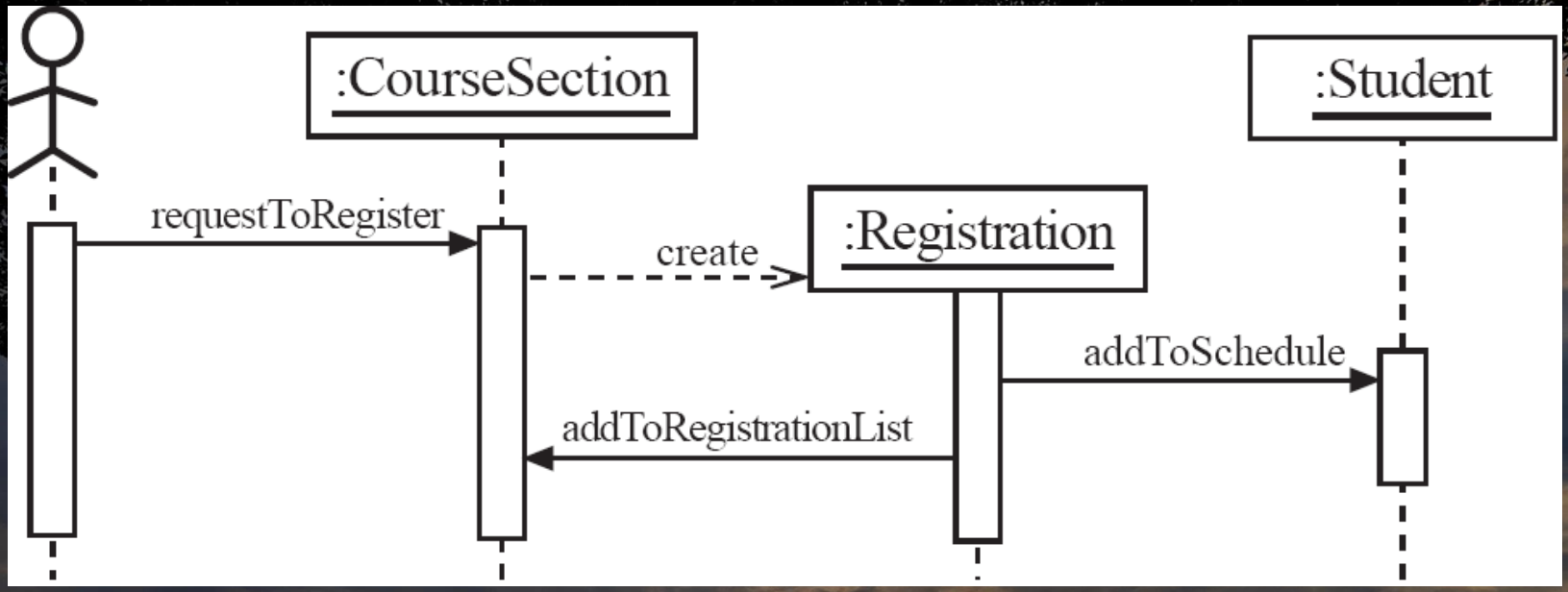

:CourseSection
:Student
requestToRegister
create
:Registration
addToSchedule
addToRegistrationList

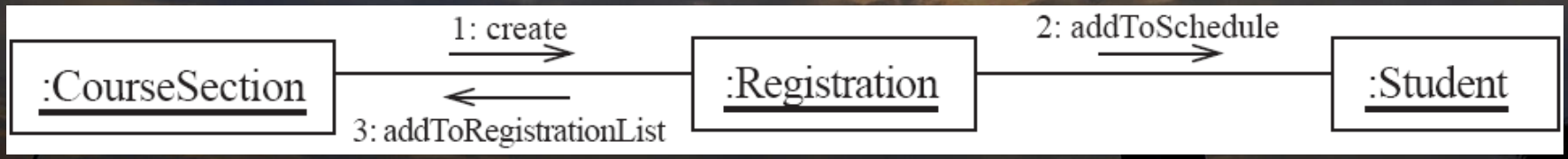

:GUI
:CourseSection
aStudent: Student
:Course
requestToRegister
requestToRegister
(aStudent)
prereq :=
getPrerequisite
hasPrerequisite :=
hasPassedCourse(prereq)
opt
[hasPrerequisite]
create
:Registration
addToSchedule
addToRegistrationList

UML collaboration diagram

- A collaboration diagram emphasizes the relationship of the objects participating in an interaction (a.k.a. communication diagram)
- They are very similar to sequence diagrams - sometimes considered as a cross between class and sequence diagram
- Unlike a sequence diagram, you don’t have to show the lifeline of an object in a collaboration diagram
- The sequence of events are indicated by sequence numbers preceding messages - therefore, a collaboration diagram can be considered as an extension of an object diagram
    - In addition to the associations among objects, collaboration diagram shows the messages the objects send each other.

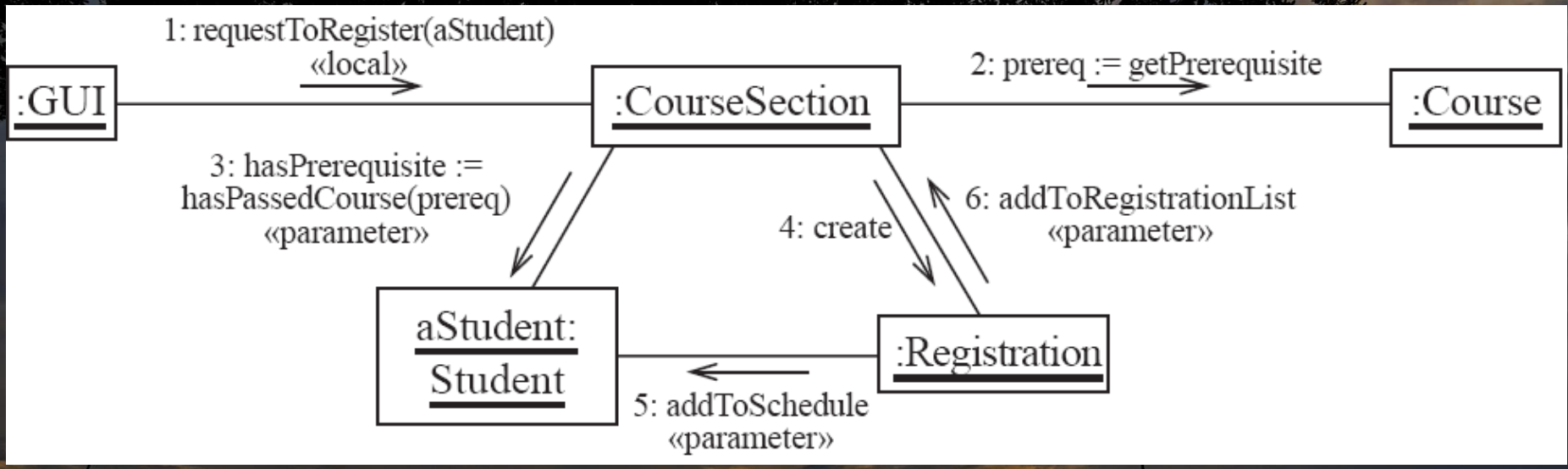

: Order Entry Window
Object
Sequence Number
1: prepare()
Message
: Order
Self-Delegation
5: needToReorder()
2*: prepare()
3: check()
4: [check == true] remove()
: Order Line
: Stock Item
7: [check == true] new
6: new
:Delivery Item
:Reorder Item


:CourseSection
:Student
requestToRegister
create
:Registration
addToSchedule
addToRegistrationList


1: create
2: addToSchedule
:CourseSection
:Registration
:Student
3: addToRegistrationList

Collab. diagram messages

- Messages among objects are represented with arrows that points to receiving object, near the association line between two objects.
- A label near the arrow shows what the message is with pair of parentheses ends the message
- Parameters are specified within parentheses if necessary
- Messages can occur in different object communication scenarios
  - Most commonly, the classes of the two objects have an *association* between them and thus communicate messages
  - The receiving object is stored in a *local* variable of the sending method → indicated as <<local>> or [L]
  - A reference to the receiving object has been received as a *parameter* of the sending method → <<parameter> or [P]
  - The receiving object is global i.e. using static method → <<global>> or [G]
  - Objects communicate over network → <<network>> or [N]


1: requestToRegister(aStudent)
«local»
:GUI
:CourseSection
2: prereq := getPrerequisite
:Course
3: hasPrerequisite :=
hasPassedCourse(prereq)
«parameter»
4: create
6: addToRegistrationList
«parameter»
aStudent:
Student
:Registration
5: addToSchedule
«parameter»

Sequence vs Collaboration

- Both the collaboration diagram and sequence diagram derive from the same information in the UML's metamodel
- So you can take a diagram in one form and convert it into the other since they are semantically equivalent
- Why use sequence diagram?
  - To make explicit the time ordering of the interaction.
    - Use cases make time ordering explicit too
    - So sequence diagrams are a natural choice when you build an interaction model from a use case.
  - Make it easy to add details to messages
    - Communication diagrams have less space for this
- Why collaboration diagram?
    - Might be preferred when you are deriving an interaction diagram from a class diagram.
    - Are also useful for validating class diagrams

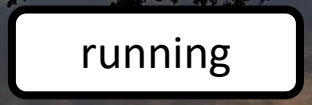
UML state diagrams

- A state diagram describes the behaviour of [1] a system, [2] some part of a system or [3] an individual object
- At any given point in time, the system or object is in a certain state
- Being in a state means that it is will behave in a specific way in response to any events that occur
- Events will cause the system to change state
- In the new state, the system will behave in a different way to events

- The state diagram is a directed graph where the nodes are states and the arcs are transitions that indicate events.

State diagram states

- A state diagram may represent multiple states that a system can be in but at any given point in time, the system is in one state only
- It will remain in this state until an event occurs that causes it to change state
- A state is represented by a rounded rectangle containing the name of the state.



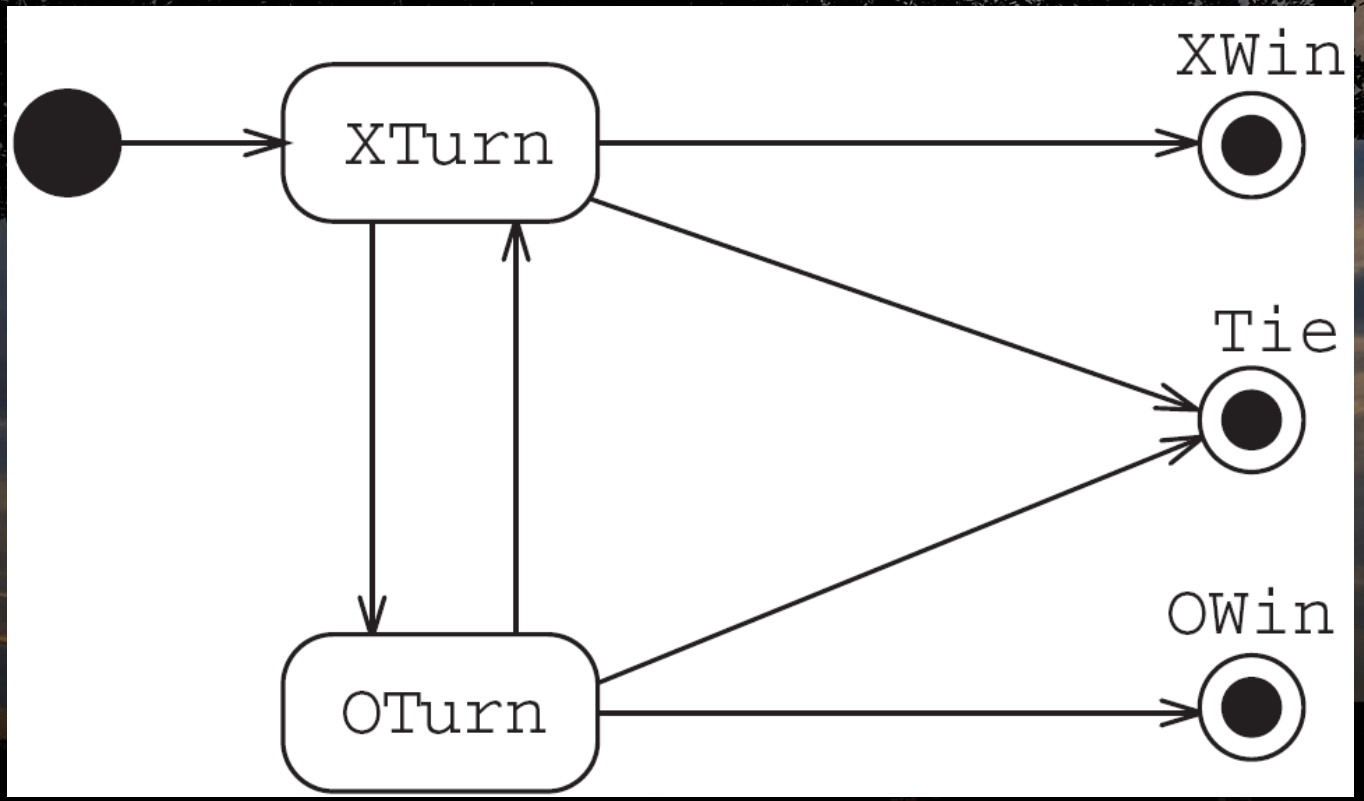
- Special states:
  - A black circle represents the start state
  - A circle with a ring around it represents an end state

State diagram transitions

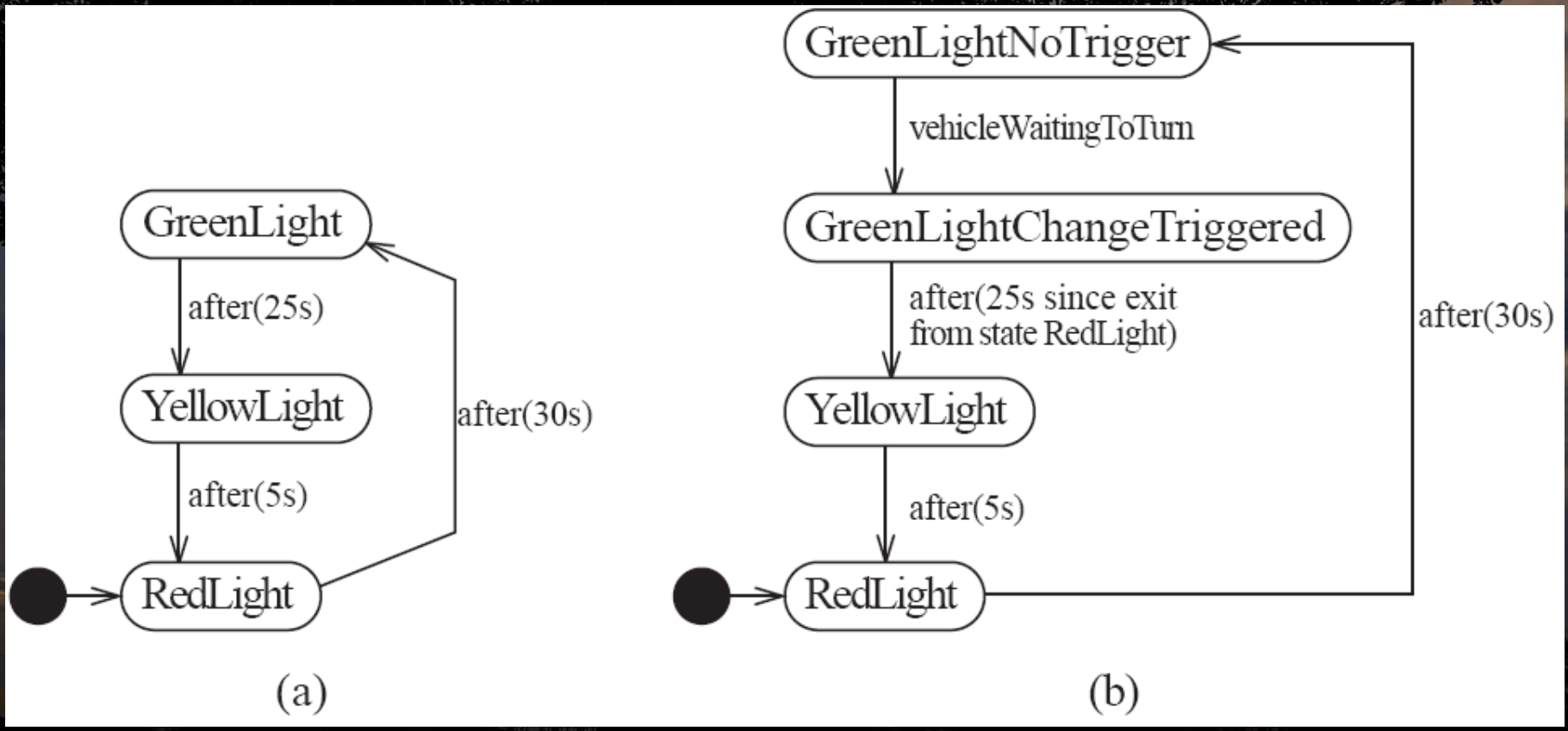
- A transition represents a change of state in response to an event
- It is considered to occur instantaneously
- The label on each transition is the event that causes the change of state
- Represented with directed arrows from the source state to destination state
- State diagrams may make use of conditions and branching (similar to interaction diagrams) to indicate different transitions
    - State diagrams can also indicate time-outs which are events that occur on their own upon the end of a timer

Example

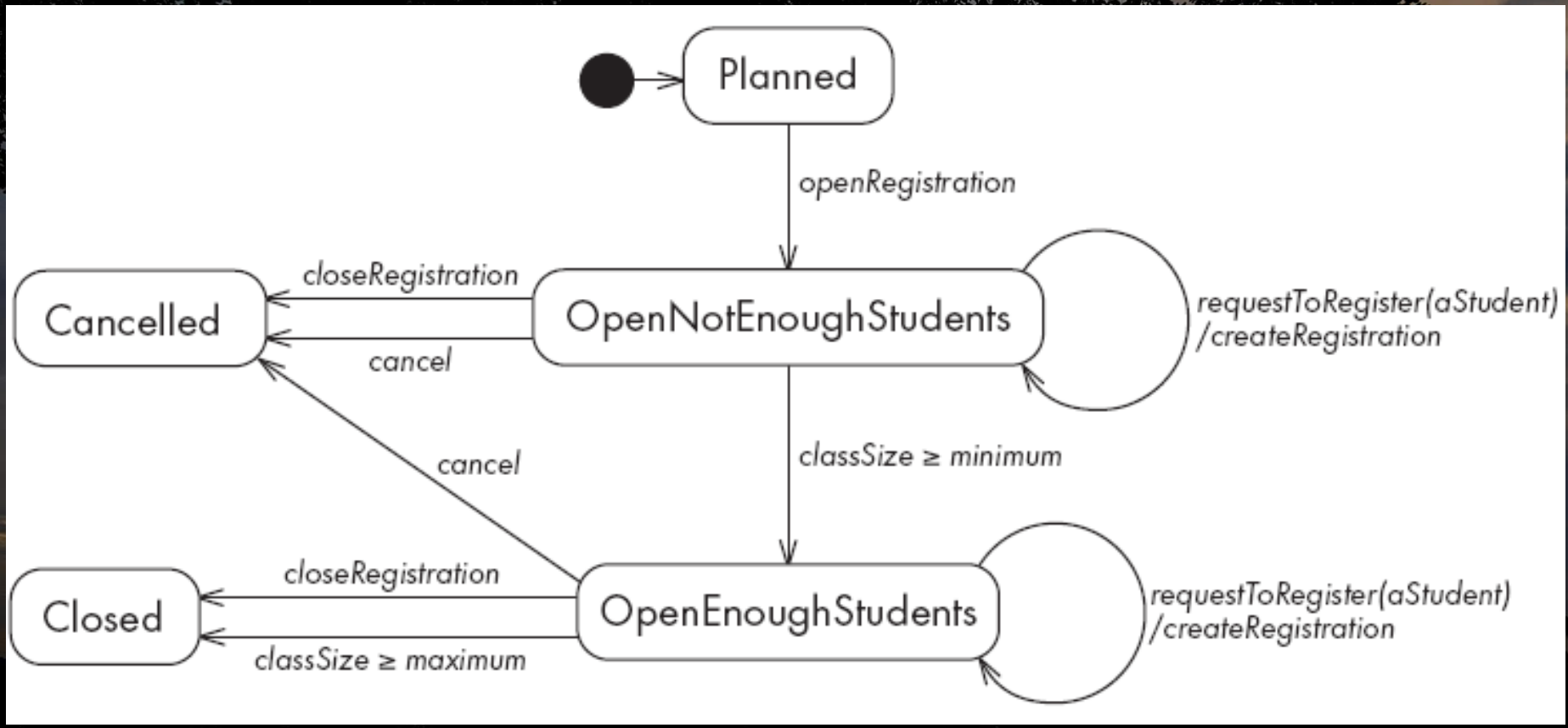
- Tic-tac-toe example



Example with labeled transitions



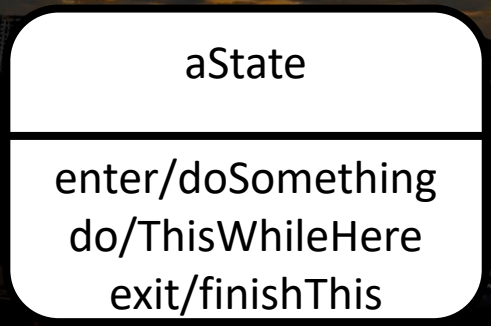
Example with conditional transitions



State diagram activities

- An activity is something that takes place while the system is in a particular state
  - It occurs in a specific a period of time.
  - The system may take a transition out of the state in response to completion of the activity,
  - Some other outgoing transition may result in:
    - The interruption of the activity, and/or
    - An early exit from the state

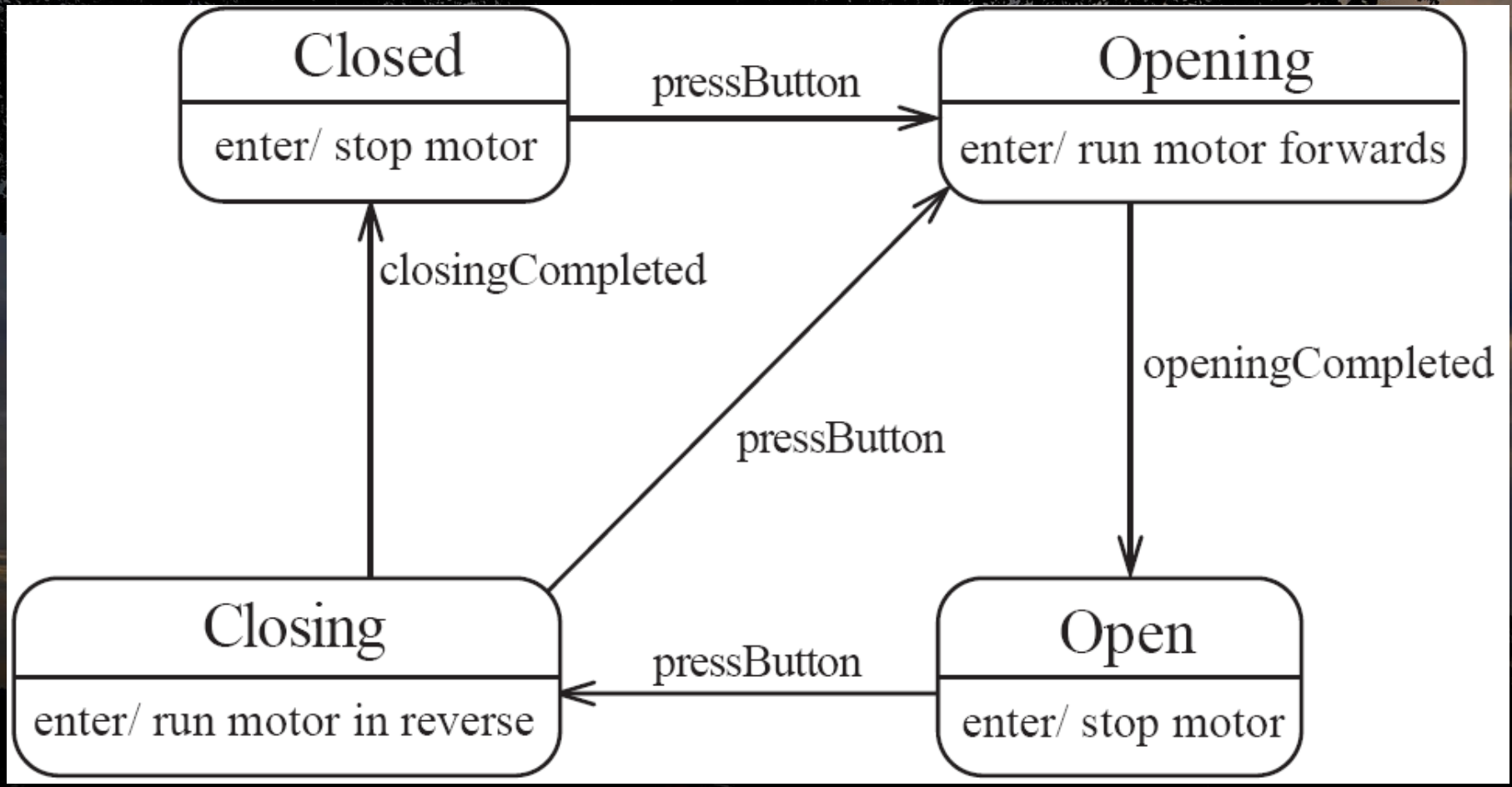
- Activities take form in the diagram with an extended state notation

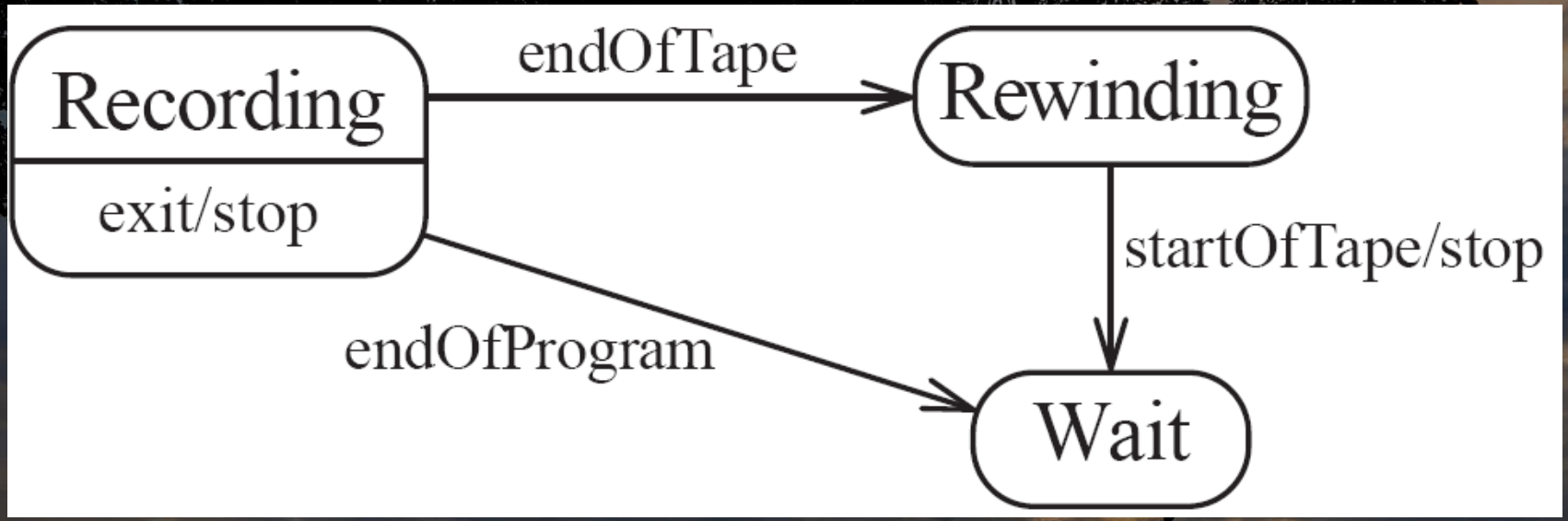


Actions in SD activities

- Activities in a state diagram state map into the functions that will appear in the finalised program
- An action is something that takes place effectively instantaneously
    - When a particular transition is taken
    - Upon entry into a particular state
    - Upon exit from a particular state
- An action should consume no noticeable amount of time
- Usually listed as a short description of the actual activity

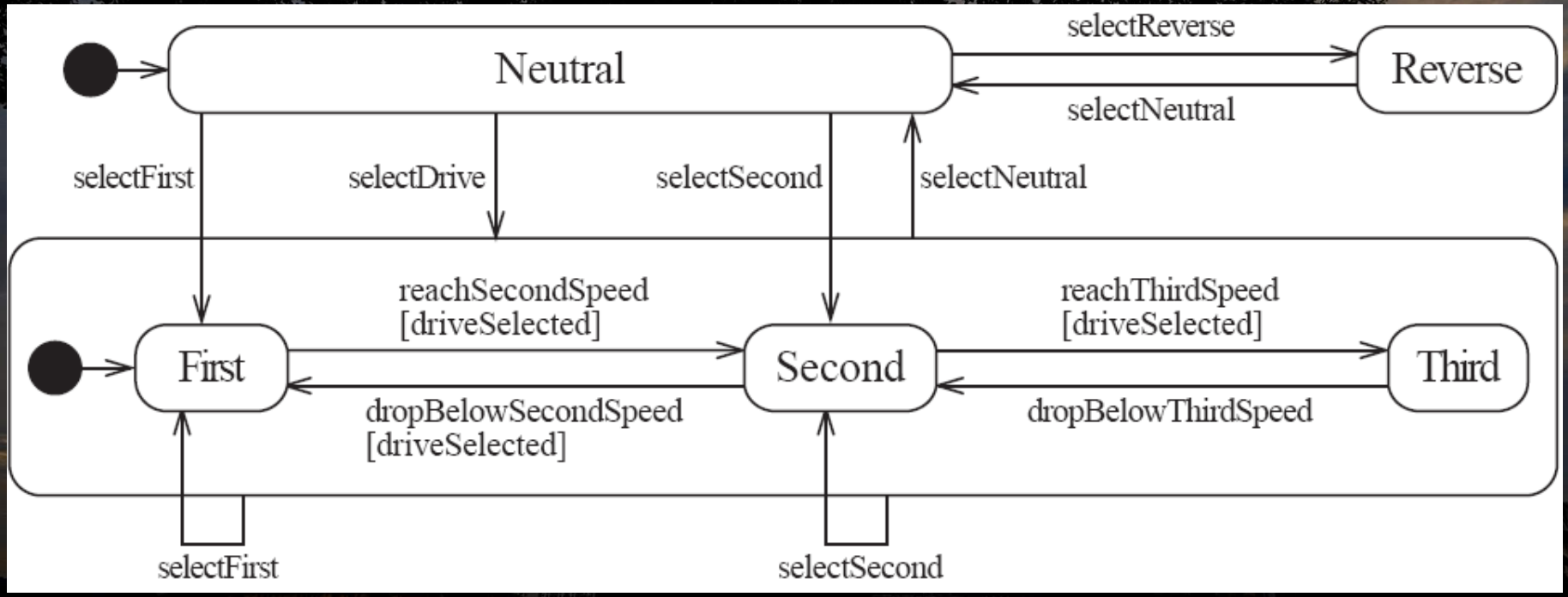
Example with activities/actions

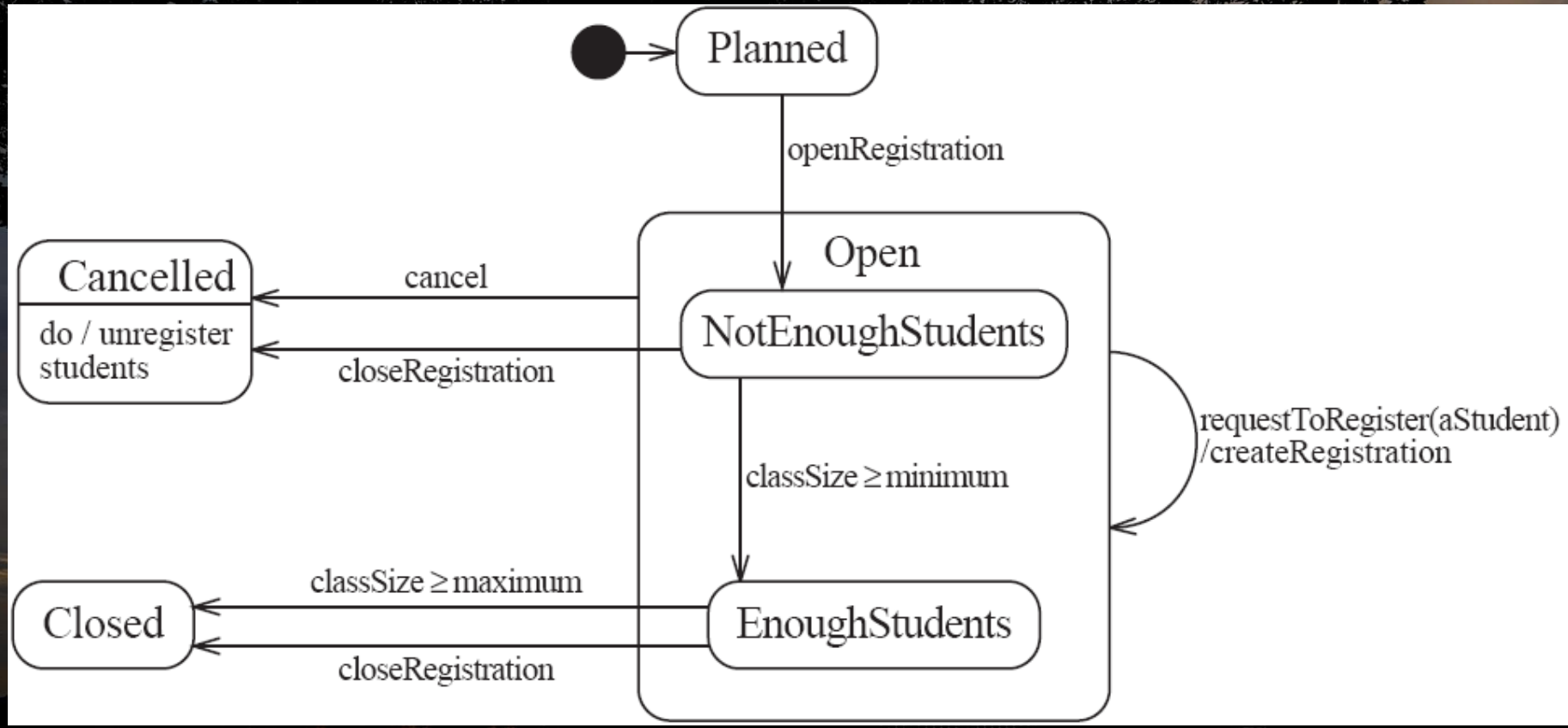



Recording
exit/stop
endOfTape
Rewinding
startOfTape/stop
endOfProgram
Wait

Nested state diagrams

- State diagrams can have nested state diagrams → the states in the inner diagram are called *substates*




Planned
openRegistration
Open
Cancelled
do / unregister students
cancel
closeRegistration
NotEnoughStudents
requestToRegister(aStudent)
/createRegistration
classSize ≥ minimum
classSize ≥ maximum
Closed
EnoughStudents
closeRegistration

UML activity diagram

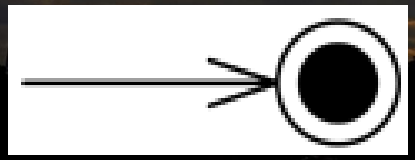
- An activity diagram is like a state diagram - except most transitions are caused by internal events, such as the completion of a computation
- An activity diagram
    - Can be used to understand the flow of work that an object or component performs
    - Can also be used to visualize the interrelation and interaction between different use cases
    - Is most often associated with several classes
- One of the strengths of activity diagrams is the representation of concurrent activities
- Activity diagrams use the same major elements from a state diagram but with additions to represent branching and concurrency
- Transitions are often called edges in official documents and refers to flows in the activity diagram

Flows

- Start (initial) node - Control node at which flow starts when the activity is invoked

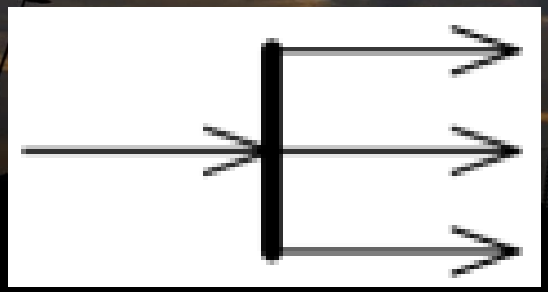
- Flow final node - control final node that terminates a flow but does not affect other flows that may continue running

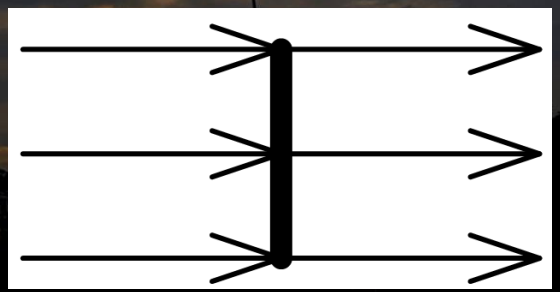
- Activity final node - control final node that stops all flows in an activity. An activity may have more than one activity final node.



Activity diagram concurrency

- Concurrency is shown using forks, rendezvous, joins, merges and decisions.
- A *fork* has one incoming flow and multiple outgoing flows - the execution splits into two concurrent threads.
- A *rendezvous* has multiple incoming and multiple outgoing flows - Once all the incoming flows occur all the outgoing flows may occur.
- The notation for a fork/rendezvous node is a thick line segment with a single/multiple activity edge entering it, and two or more edges leaving it.

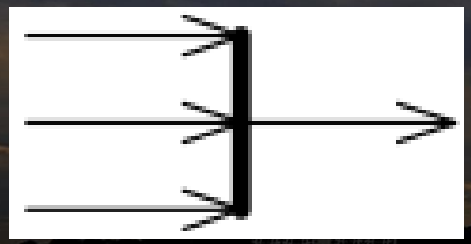


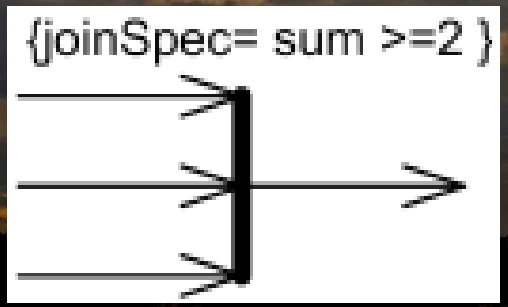


- Both fork and rendezvous support parallelism in activities.

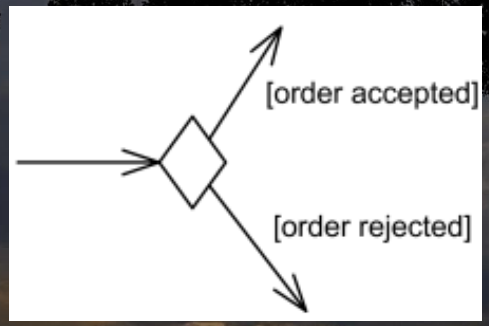
Source:http://www.uml-diagrams.org/activity-diagrams-controls.html

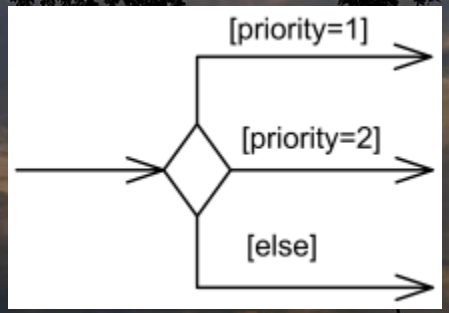
- A *Join* node is a control node that has multiple incoming flows and one outgoing flow and is used to synchronize incoming concurrent flows.
- Join nodes are introduced to support parallelism in activities
- The default action of any join node is the AND all the incoming activities.
- If a different action is required, then tags/labels in curly braces can be added to identify the condition of the join action
- The notation symbol for join is the same as the fork



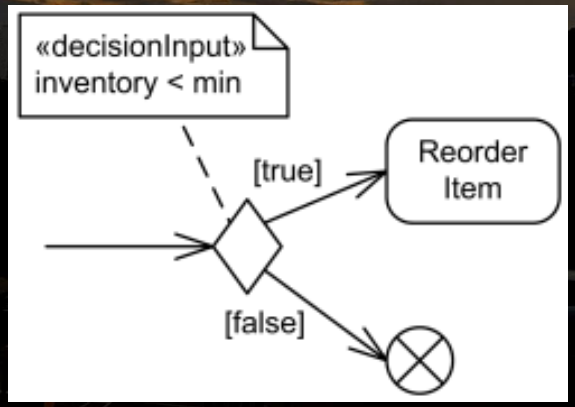


- *Decision* nodes are control nodes that accept one or two incoming edges (flows) and selects one outgoing edge from one or more outgoing flows - Each flow is independent of each other and only one path is taken upon the decision
- The notation for a decision node is a diamond-shaped symbol
- For decision points, a predefined guard "else" may be defined for at most one outgoing edge

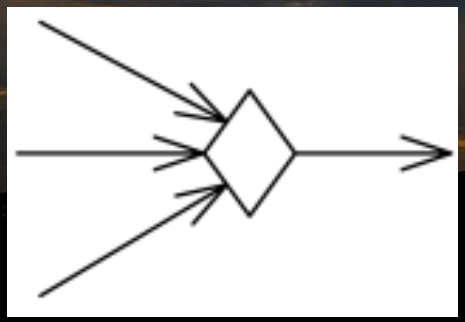


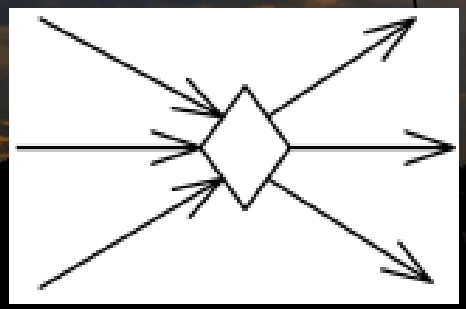


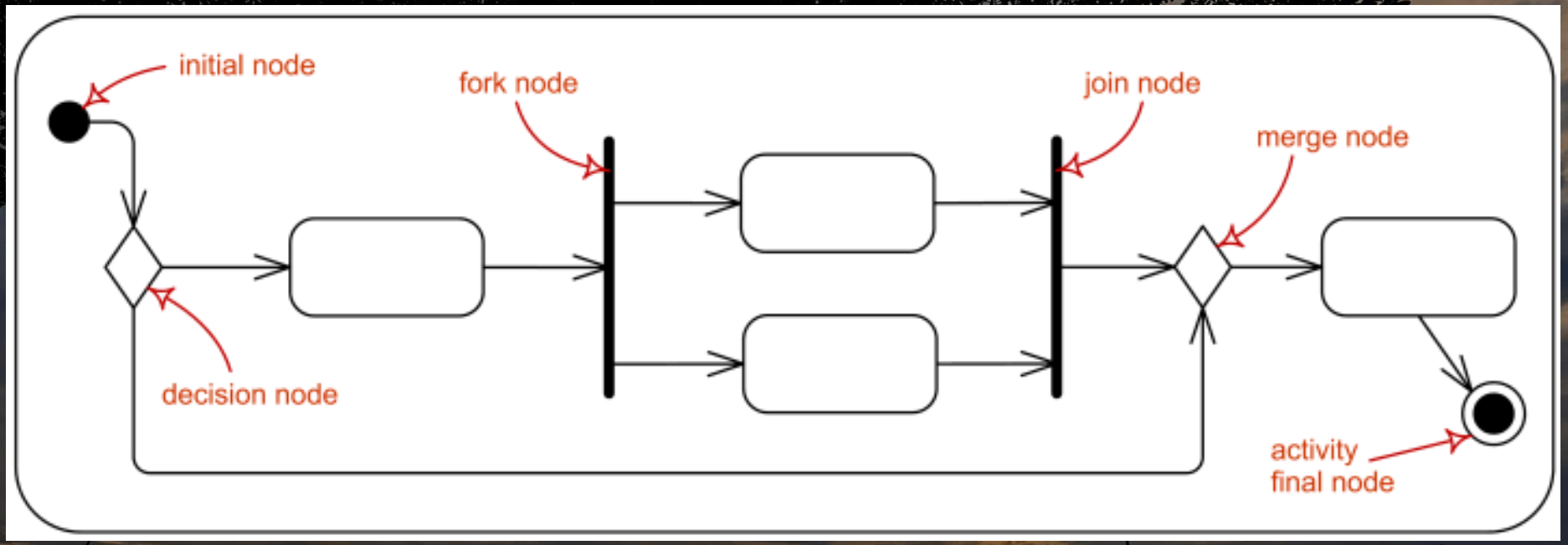
- Decisions can be made and indicated with input behaviour



- *Merge* nodes brings together multiple incoming alternate flows to accept single outgoing flow
- Merge can/should not be used to synchronize concurrent flows
- Merge actions occur after a decision and not after a fork (and vice-versa applies)
- The notation for a merge node is a diamond-shaped symbol with two or more edges entering it and a single activity edge leaving it
- A decision can immediately follow a merge and be represented with just a single diamond (or alternatively two diamonds if clarity is preferred)

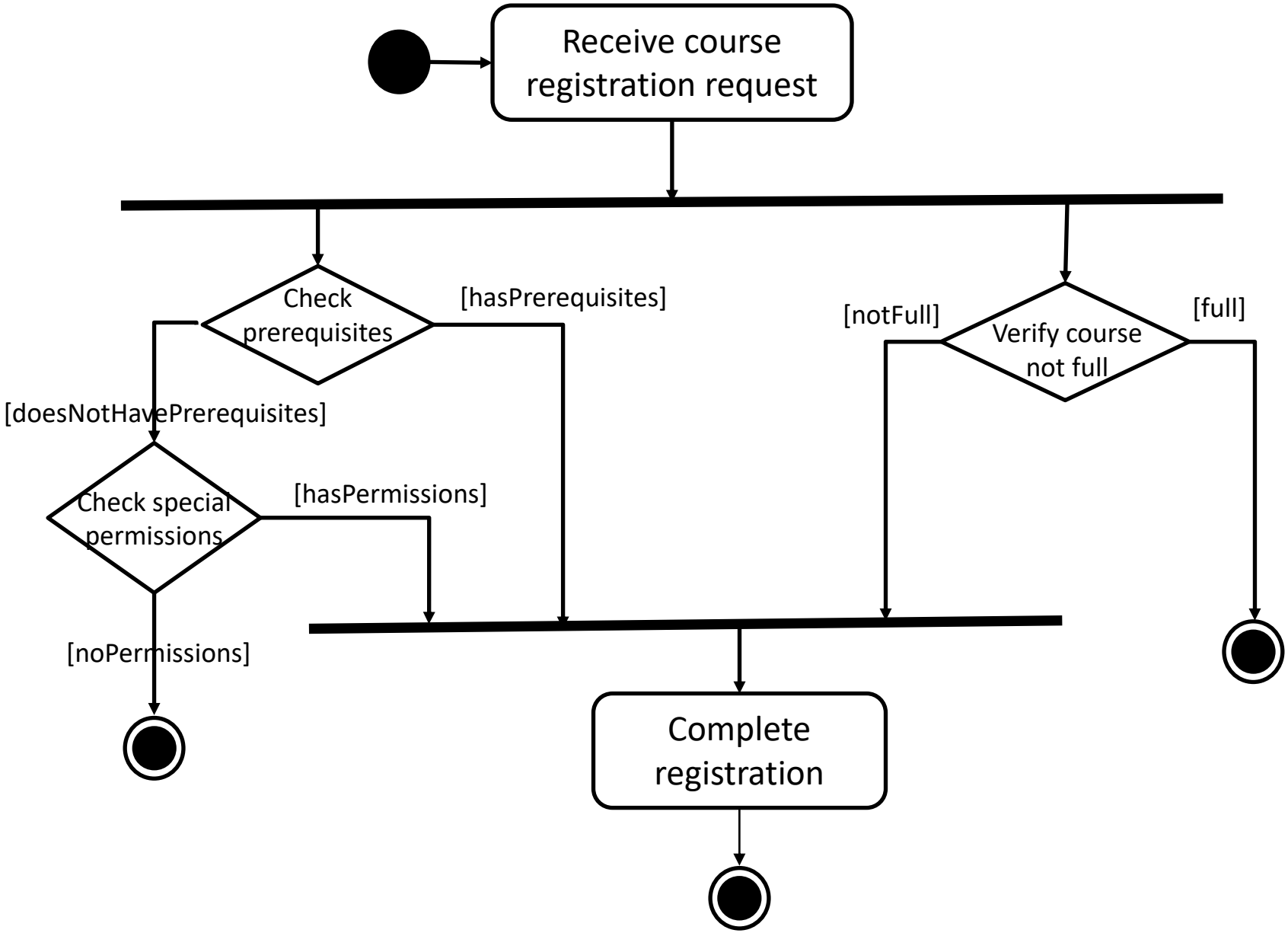






Source: http://www.uml-diagrams.org/activity-diagrams-controls.html

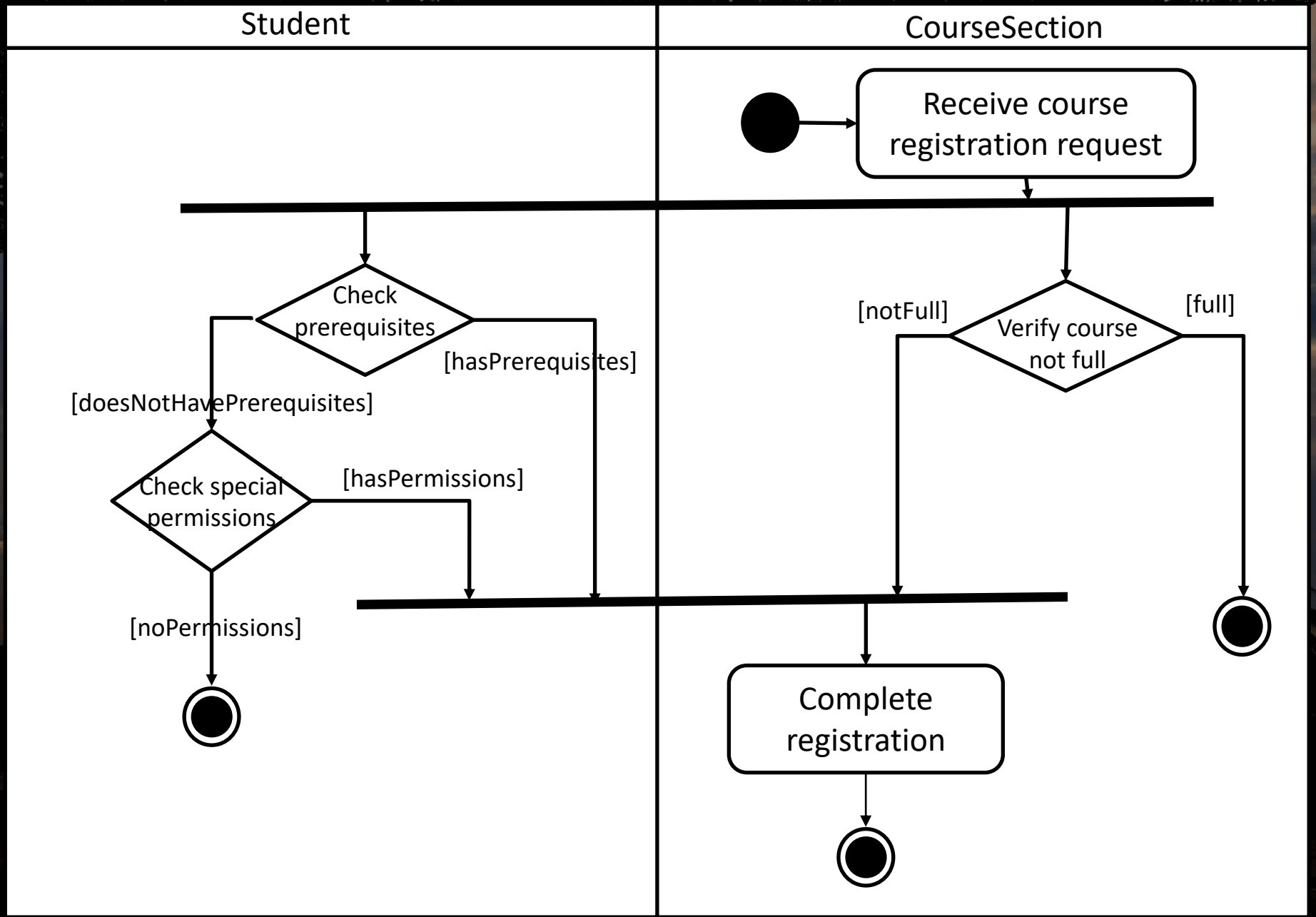
Example


Receive course registration request
Check prerequisites
[hasPrerequisites]
[doesNotHavePrerequisites]
Check special permissions
[hasPermissions]
[noPermissions]
[notFull]
Verify course not full
[full]
Complete registration

Activity diagram swimlanes

- Activity diagrams are most often associated with several classes.
- The partition of activities among the existing classes can be explicitly shown using swimlanes
- Each swimlane identifies which portion of the current activity occurs in the related classes – especially important for classes that communicate with many other classes
- Swimlanes are indicated with boxes that surround the functions related to a particular class.

AD swimlane example

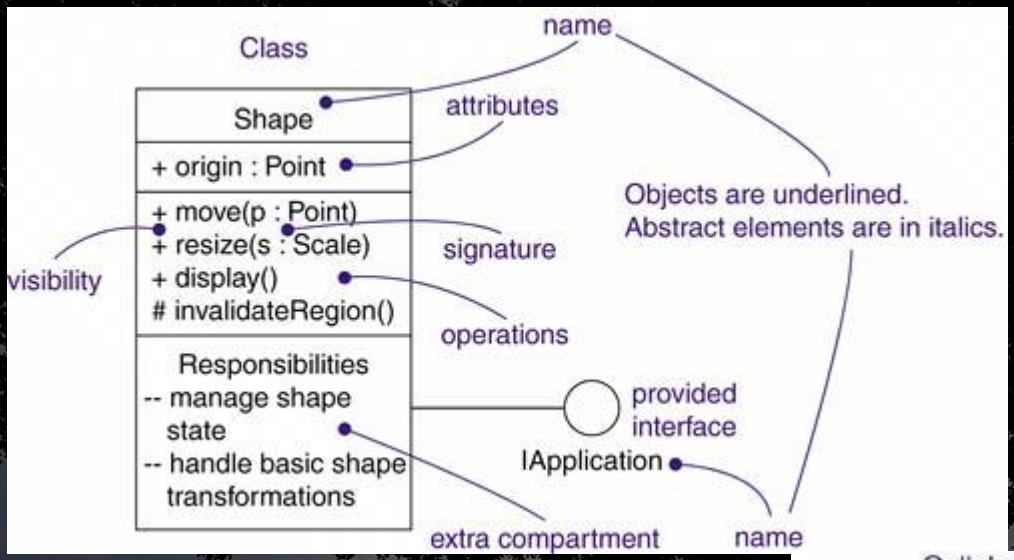


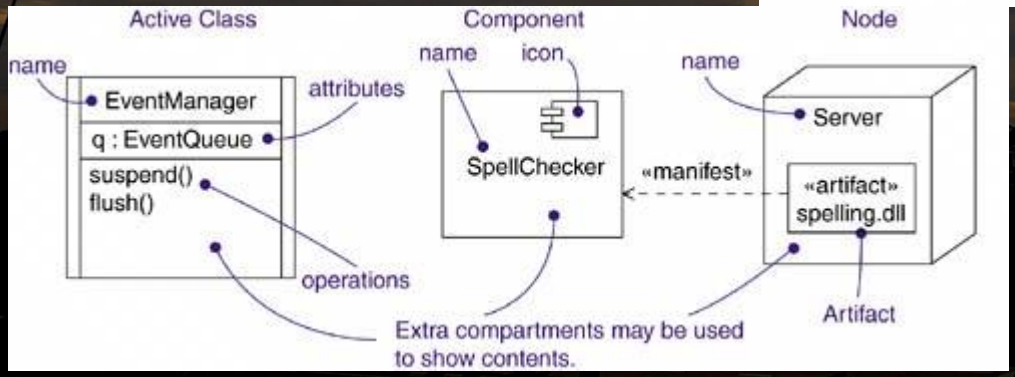
Risks in modelling

- Dynamic modelling is a difficult skill
- In a large system there are a very large number of possible paths a system can take.
- It is hard to choose the classes to which to allocate each behaviour - Ensure that skilled developers lead the process, and ensure that all aspects of your models are properly reviewed.
- Work iteratively
    - Develop initial class diagrams, use cases, responsibilities, interaction diagrams and state diagrams;
    - Then go back and verify that all of these are consistent, modifying them as necessary.
- Drawing different diagrams that capture related, but distinct, information will often highlight problems.

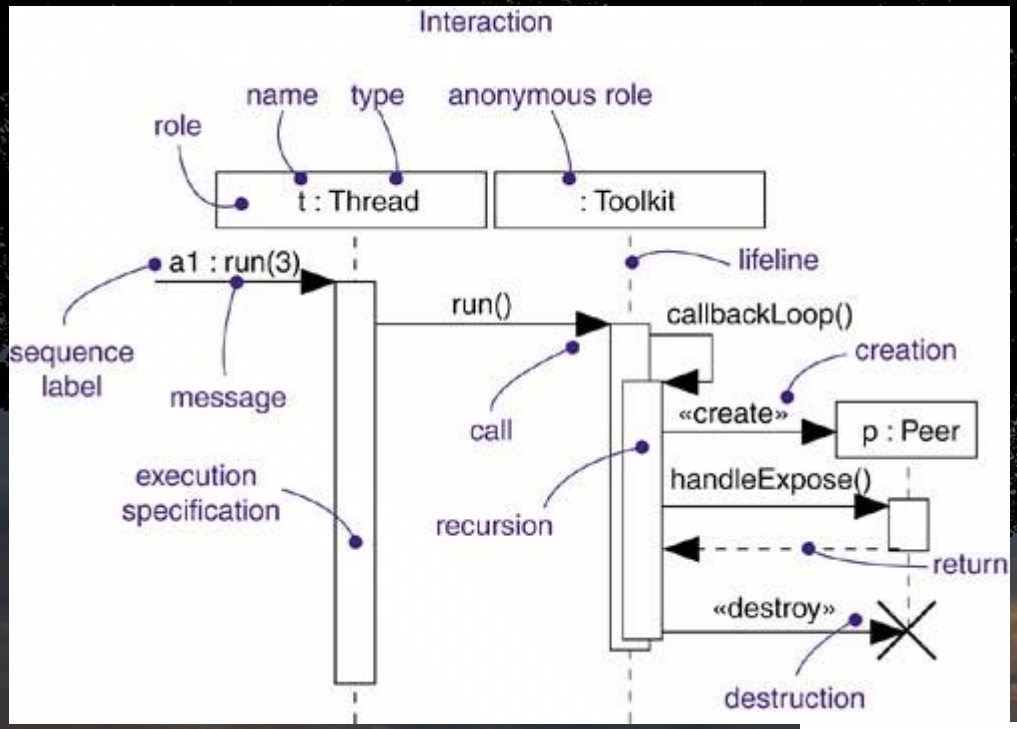
Summary

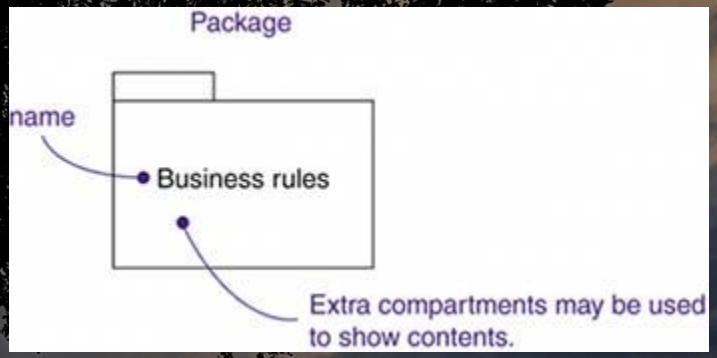
UML diagram	Used for
Use case diagram	• Used to capture the behaviours of a system. • It consists of use cases, actors and their relationships. • Use case diagram is used at a high level design to capture the requirements of a system. • So it represents the system functionalities and their flow
Interaction diagram	• Used for capturing dynamic nature of a system. • Sequence and collaboration diagrams used for this purpose. • Sequence diagrams are used to capture time ordering of message flow and collaboration diagrams are used to understand the structural organization of the system.
State diagram	• State diagrams are one of the five diagrams used for modeling dynamic nature of a system. • These diagrams are used to model the entire life cycle of an object.
Activity diagram	• Activity diagram consists of activities, links, relationships etc. • It models all types of flows like parallel, single, concurrent etc. • Activity diagram describes the flow control from one activity to another without any messages. • These diagrams are used to model high level view of business requirements

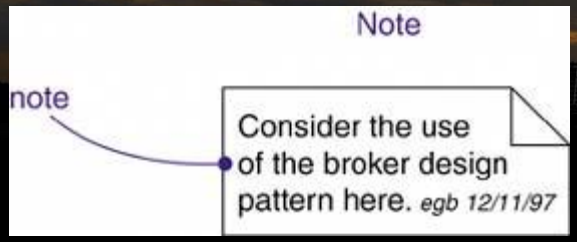


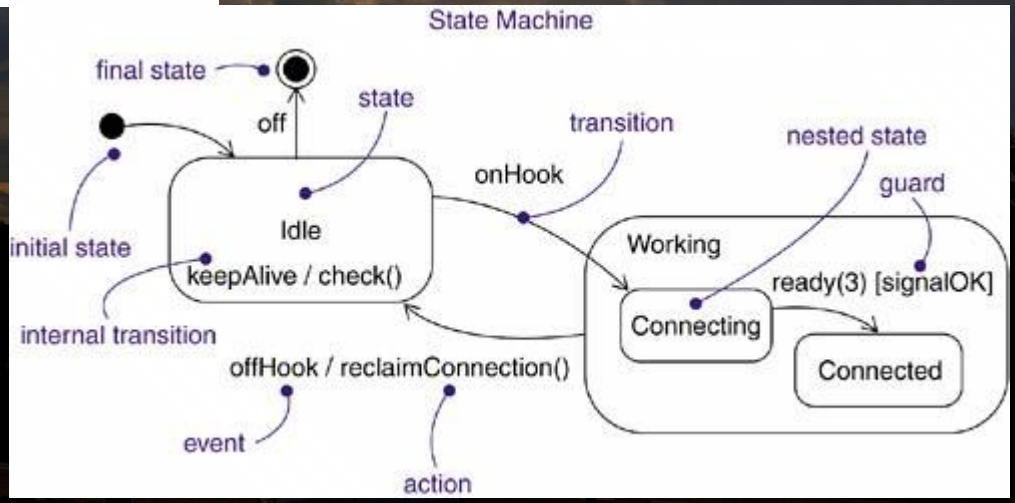


Source: http://umlguide2.uw.hu/app01lev1sec1.html


Interaction
name
type
anonymous role
role
t : Thread
: Toolkit
lifeline
a1 : run(3)
run()
callbackLoop()
sequence label
message
call
creation
«create»
p : Peer
execution specification
recursion
handleExpose()
return
«destroy»
destruction


Package
name
Business rules
Extra compartments may be used to show contents.


Note
note
Consider the use of the broker design pattern here. egb 12/11/97


State Machine
final state
state
off
transition
nested state
onHook
guard
initial state
Idle
Working
keepAlive / check()
ready(3) [signalOK]
internal transition
Connecting
Connected
offHook / reclaimConnection()
event
action